OpenClaw

代码结构深度分析文档

Comprehensive Code Architecture Analysis

项目名称：	OpenClaw — 开源个人 AI 助手平台
版本：	v2026.3.9
仓库地址：	https://github.com/openclaw/openclaw
许可证：	MIT License
核心语言：	TypeScript (ESM) / Python / Swift / Kotlin
代码规模：	~897,000 行 / 1,500+ 源文件 / 77 子模块
分析日期：	2026 年 3 月 9 日

彦纾 & 睿鸿

wys17@tsinghua.org.cn zhihong.lr@alibaba-inc.com

欢迎随意传播，我们支持技术的自由化

2026 年 3 月

本文档基于 OpenClaw 2026 年 3 月版本（v2026.3.x）编写
涵盖网关协议、渠道集成、Agent 运行时、插件系统、Hook 事件、
Memory 向量搜索、配置管理、安全体系、前端 UI、原生应用、
扩展生态、技能系统、测试架构、CI/CD 流水线等 25 章内容

Contents

1 项目概述与背景 **2**
- 1.1 项目定位 . . . 2
- 1.2 核心愿景 . . . 2
- 1.3 项目版本与许可 . . . 2
- 1.4 项目规模统计 . . . 3
- 1.5 社区与贡献模型 . . . 3

2 技术栈与依赖分析 **4**
- 2.1 编程语言 . . . 4
- 2.2 运行时与构建 . . . 4
- 2.3 TypeScript 编译配置 . . . 4
- 2.4 核心运行时依赖 . . . 4
  - 2.4.1 基础框架 . . . 5
  - 2.4.2 AI/ML 相关 . . . 5
  - 2.4.3 消息渠道 SDK . . . 5
  - 2.4.4 UI 框架 . . . 5
  - 2.4.5 媒体处理 . . . 6
- 2.5 代码质量工具链 . . . 6

3 Monorepo 目录结构总览 **7**
- 3.1 顶层目录 . . . 7
- 3.2 Workspace 配置 . . . 8
- 3.3 src/ 目录子模块分类 . . . 8
  - 3.3.1 CLI 与命令 . . . 8
  - 3.3.2 网关与通信 . . . 8
  - 3.3.3 智能体与 AI . . . 8
  - 3.3.4 基础设施 . . . 9
  - 3.3.5 插件与扩展 . . . 9
  - 3.3.6 数据与记忆 . . . 9
  - 3.3.7 媒体与理解 . . . 9
  - 3.3.8 高级功能 . . . 9
  - 3.3.9 通用支持 . . . 10
- 3.4 package.json 导出映射 . . . 10

4 核心架构设计与数据流 **11**
- 4.1 网关中心型架构 . . . 11
- 4.2 分层架构 . . . 11
- 4.3 消息完整数据流 . . . 11
- 4.4 五大设计原则 . . . 12
- 4.5 关键子系统交互 . . . 13

5 程序入口与启动流程 **14**
- 5.1 入口文件：entry.ts . . . 14
- 5.2 模块导出：index.ts . . . 14
- 5.3 CLI 命令树：cli/program.ts . . . 14
- 5.4 网关启动时序 . . . 15
- 5.5 依赖注入模式 . . . 16

6 WebSocket 网关服务器 **18**
- 6.1 核心实现概述 . . . 18
- 6.2 运行时状态结构 . . . 18
- 6.3 WebSocket 连接管理 . . . 18

6.3.1 连接附加 . . . 18
6.3.2 消息处理器 . . . 19
6.4 广播机制 . . . 19
6.5 多绑定主机 . . . 19
6.6 Canvas Host 集成 . . . 19
6.7 优雅关闭 . . . 20

7 网关协议与帧结构 21
7.1 协议概述 . . . 21
7.2 帧类型 . . . 21
7.3 帧结构定义 . . . 21
7.4 AJV 验证器 . . . 21
7.5 ConnectParams 握手参数 . . . 22
7.6 ChatSendParams 聊天发送参数 . . . 22
7.7 协议版本管理 . . . 22

8 RPC 方法分发与处理器 23
8.1 方法聚合器 . . . 23
8.2 请求处理完整流程 . . . 23
8.3 请求上下文 . . . 24
8.4 主要处理器组详解 . . . 24
8.4.1 chat 处理器 . . . 25
8.4.2 chat.send 投递路由 . . . 25
8.4.3 config 处理器 . . . 25
8.4.4 其他关键处理器 . . . 25
8.5 插件方法扩展 . . . 25

9 认证、授权与 RBAC 27
9.1 认证体系概述 . . . 27
9.2 设备签名验证 . . . 27
9.2.1 Ed25519 签名 . . . 27
9.2.2 时间戳偏差容忍 . . . 27
9.3 角色与权限模型 . . . 27
9.3.1 角色定义 . . . 27
9.3.2 三层授权检查 . . . 27
9.4 速率限制 . . . 28
9.4.1 IP 速率限制 . . . 28
9.4.2 控制面写入预算 . . . 28
9.5 静默本地配对 . . . 29
9.6 浏览器来源验证 . . . 29

10 渠道集成架构 30
10.1 ChannelPlugin 泛型类型 . . . 30
10.2 渠道能力矩阵 . . . 30
10.3 渠道账户快照 . . . 30

11 渠道适配器详解 33
11.1 适配器组合模式 . . . 33
11.2 ChannelConfigAdapter — 配置适配器 . . . 33
11.3 ChannelSecurityAdapter — 安全适配器 . . . 33
11.4 ChannelOutboundAdapter — 出站适配器 . . . 34
11.5 ChannelSetupAdapter — 设置适配器 . . . 34
11.6 其他适配器 . . . 35
11.7 Telegram 渠道实现深度分析 . . . 35

11.7.1 能力声明 . . . 35
11.7.2 出站配置 . . . 35
11.7.3 安全策略 . . . 35
11.7.4 多账户支持 . . . 36

12 消息路由与会话管理 37
12.1 会话键（SessionKey） . . . 37
12.2 入站路由流程 . . . 37
12.3 出站路由 . . . 37
12.4 会话存储 . . . 38
12.4.1 JSONL 格式 . . . 38
12.4.2 DAG 消息结构 . . . 38
12.5 会话类型 . . . 39
12.6 session-utils.ts 工具集 . . . 39
12.7 并发控制 . . . 39
12.7.1 KeyedAsyncQueue . . . 39
12.7.2 去重机制 . . . 39

13 消息投递队列系统 41
13.1 架构设计 . . . 41
13.2 QueuedDelivery 完整结构 . . . 41
13.3 原子性模型 . . . 41
13.4 指数退避重试 . . . 42
13.5 bestEffort 模式 . . . 42
13.6 镜像通知 . . . 42
13.7 心跳过滤 . . . 43

14 Agent 智能体系统 44
14.1 子系统概览 . . . 44
14.2 Agent 作用域解析 . . . 44
14.3 多智能体工作区隔离 . . . 44
14.4 Auth Profile 管理 . . . 44
14.5 Pi 嵌入式运行器 . . . 45
14.6 上下文引擎 . . . 46
14.7 自动回复系统 . . . 46
14.8 模型提供商 . . . 46

15 工具系统与执行引擎 47
15.1 工具类型定义 . . . 47
15.2 工具分类 . . . 47
15.3 浏览器自动化 . . . 47
15.4 HTTP 工具调用 . . . 48
15.5 工具事件接收器 . . . 48
15.6 执行审批 . . . 48
15.7 Canvas Host . . . 49
15.8 Node Host . . . 49

16 插件系统架构 50
16.1 插件定义 . . . 50
16.2 八种注册类型 . . . 50
16.3 插件注册表 . . . 50
16.4 动态加载 . . . 51
16.5 插件运行时 . . . 51
16.6 Plugin SDK . . . 52

16.7 插件包结构规范 52

17 Hook 事件驱动系统 **53**
17.1 Hook 执行框架 53
17.2 完整生命周期事件 53
17.3 Hook 优先级 54
17.4 Hook 来源分类 54
17.5 内置 Hook 详解 54
17.5.1 boot-md Hook 54
17.5.2 session-memory Hook 55
17.5.3 command-logger Hook 55
17.6 错误处理 55

18 Memory 记忆与向量搜索 **56**
18.1 类继承层次 56
18.2 MemorySearchManager 接口 56
18.3 MemoryIndexManager 实现 56
18.4 嵌入向量提供商 57
18.5 存储层 57
18.6 混合搜索 58
18.7 同步触发器 58

19 配置管理与热重载 **59**
19.1 配置加载 59
19.2 ConfigFileSnapshot 结构 59
19.3 JSON5 配置格式 59
19.4 配置层次 60
19.5 Zod Schema 验证 60
19.6 热重载机制 60
19.6.1 文件监听 60
19.6.2 重载计划构建 61
19.6.3 递归差异比较 61
19.6.4 重载状态机 61

20 安全机制与凭据管理 **62**
20.1 安全体系总览 62
20.2 凭据管理 62
20.2.1 存储结构 62
20.2.2 secrets/apply.ts 凭据申请 62
20.3 DM 策略 63
20.4 白名单验证 63
20.5 路径边界强制 64
20.6 技能权限扫描 64
20.7 自动修复 64

21 前端 UI 系统 (Lit Web Components) **65**
21.1 技术选型 65
21.2 主应用组件 65
21.3 控制器层 66
21.4 功能标签页 66
21.5 WebSocket 实时通信 66

22 原生应用 (macOS/iOS/Android) **68**
22.1 macOS 应用 68

22.1.1 技术栈 68
22.1.2 AppState 核心状态 68
22.1.3 核心组件 68
22.1.4 功能特点 68
22.2 iOS 应用 69
22.3 Android 应用 69
22.3.1 架构 69
22.3.2 核心界面 69
22.3.3 独特功能 69
22.4 共享框架：OpenClawKit 70

23 扩展生态与技能系统 71
23.1 官方扩展分类 71
23.1.1 消息渠道扩展（20 个） 71
23.1.2 功能扩展（21 个） 71
23.2 技能系统 72
23.2.1 技能定义格式 72
23.2.2 技能分类 72
23.2.3 Notion 技能深度分析 73
23.3 ClawHub 技能市场 73

24 测试、构建与 CI/CD 74
24.1 测试策略 74
24.2 Vitest 配置矩阵 74
24.3 测试命名约定 74
24.4 测试工具模块 74
24.5 构建流程 74
24.6 Docker 构建 75
24.6.1 多阶段构建 75
24.6.2 Docker Compose 75
24.7 CI/CD 流水线 76
24.7.1 GitHub Actions 主工作流 76
24.8 版本发布 76
24.9 守护进程管理 76

25 设计模式总结与架构评价 77
25.1 核心设计模式 77
25.2 架构亮点 77
25.3 代码质量评价 78
25.4 可维护性分析 78
25.5 扩展性评价 79
25.6 与业界最佳实践对比 79
25.7 总结 79

1 项目概述与背景

1.1 项目定位

OpenClaw 是一个开源的个人 **AI** 助手平台，采用本地优先（local-first）的设计理念，运行在用户自己的设备上。与云端 AI 助手不同，OpenClaw 的所有数据处理均在本地完成，用户拥有完全的数据控制权。项目以 MIT 许可证发布，鼓励社区贡献和二次开发。

项目的命名「OpenClaw」寓意开放（Open）与敏捷（Claw），象征着一个灵活、可扩展的 AI 交互平台。

1.2 核心愿景

根据项目的 `VISION.md` 文件，OpenClaw 的核心愿景与优先级如下：

1. 安全性：安全漏洞修复具有最高优先级，项目维护了详细的安全报告流程和信任模型边界文档。
2. 稳定性：Bug 修复和稳定性改进优先于新功能开发。
3. 多模型支持：支持主流 LLM 提供商（Anthropic、OpenAI、Google、AWS Bedrock 等），允许用户自由选择。
4. 多渠道覆盖：持续扩展消息渠道支持，目标覆盖所有主流即时通讯平台。
5. 性能优化：持续改进响应速度和资源占用。
6. 测试基础设施：保持和提升测试覆盖率，确保代码质量。
7. 易用性：降低安装和使用门槛，让非技术用户也能轻松上手。

1.3 项目版本与许可

Table 1: 项目基本信息

属性	值
项目名称	OpenClaw
当前版本	v2026.3.9
版本策略	日期语义化（vYYYY.M.D）
许可证	MIT License
仓库地址	https://github.com/openclaw/openclaw
文档站点	docs.openclaw.ai
技能市场	clawhub.ai
核心维护者	14 人

1.4 项目规模统计

Table 2: 代码库关键指标

指标	数值	说明
核心源码文件数	~1,500+	src/ 目录下
源码行数	~897,000	含注释和空行
核心子模块数	77	src/ 下的子目录
测试文件数	~300+	*.test.ts 文件
官方扩展数	41	extensions/ 目录
内置技能数	53	skills/ 目录
支持消息渠道	20+	核心 + 扩展
国际化语言	6	en, zh-CN, zh-TW, es, pt-BR, de
目标平台	5	macOS, iOS, Android, Linux, Windows
构建脚本数	114+	scripts/ 目录

1.5 社区与贡献模型

根据 `CONTRIBUTING.md`，项目采用以下贡献模型：

- **Bug 修复与小 PR**：欢迎直接提交，无需事先讨论
- 新功能：需要先在 GitHub Discussion 中讨论方案
- **AI 辅助 PR**：明确欢迎使用 AI 工具辅助开发
- 安全漏洞：有专门的快速通道（Fast-track Triage Gate）
- 提交前检查：`pnpm build && pnpm check && pnpm test`

2 技术栈与依赖分析

2.1 编程语言

Table 3: 使用的编程语言

语言	角色	使用范围
TypeScript (ESM)	主要语言	核心代码、CLI、网关、扩展、UI、插件 SDK
Python	辅助语言	技能脚本（Skills），通过 Ruff 检查，Pytest 测试
Swift	原生应用	macOS 菜单栏应用、iOS 应用、共享框架 OpenClawKit
Kotlin	原生应用	Android 应用，Gradle 构建
YAML/JSON/TOML	配置	各类配置文件、CI/CD 流水线

2.2 运行时与构建

Table 4: 运行时与构建工具链

工具	版本	用途
Node.js	$\geq$ 22.12.0	服务端运行时
pnpm	10.23.0	Monorepo 包管理器
Bun	latest	部分构建脚本加速
tsdown	latest	TypeScript 编译打包（替代 esbuild）
Vite	latest	UI 开发服务器与静态资源构建
tsx	latest	TypeScript 直接运行（开发模式）

2.3 TypeScript 编译配置

项目的 `tsconfig.json` 采用严格模式，关键配置如下：

```
{
  "compilerOptions": {
    "strict": true,                    // 启用所有严格检查
    "target": "ES2023",                // 编译目标
    "module": "NodeNext",              // ESM 模块解析
    "moduleResolution": "NodeNext",
    "experimentalDecorators": true,    // Lit 装饰器支持
    "esModuleInterop": true,
    "skipLibCheck": true,
    // 路径别名 - 用于 Plugin SDK
    "paths": {
      "openclaw/plugin-sdk/*": ["./src/plugin-sdk/*"]
    }
  }
}
```

Listing 1: tsconfig.json 核心配置

2.4 核心运行时依赖

2.4.1 基础框架

Table 5: 基础框架依赖

包名	用途
`commander`	CLI 命令行解析框架
`express`	HTTP 服务器（网关 HTTP 接口）
`ws`	WebSocket 实现（网关核心通信）
`zod`	TypeScript-first Schema 验证
`ajv`	JSON Schema 验证（协议帧验证）
`chokidar`	文件系统监听（配置热重载）
`json5`	JSON5 解析器（支持注释的配置文件）

2.4.2 AI/ML 相关

Table 6: AI/ML 依赖

包名	用途
`@anthropic-sdk/sdk`	Anthropic Claude API 客户端
`@aws-sdk/client-bedrock`	AWS Bedrock 多模型接入
`@mariozechner/pi-*`	Pi 嵌入式 Agent 运行时库
`sqlite-vec`	SQLite 向量扩展（本地向量数据库）

2.4.3 消息渠道 SDK

Table 7: 渠道集成 SDK

包名	对应渠道
`grammy`	Telegram Bot API
`@slack/bolt`	Slack App
`discord-api-types`	Discord Bot
`@whiskeysockets/baileys`	WhatsApp Web
`@line/bot-sdk`	LINE Messaging API
`@larksuiteoapi/node-sdk`	飞书 (Feishu/Lark)

2.4.4 UI 框架

Table 8: 前端 UI 依赖

包名	用途
`lit`	Web Components 框架
`@lit/context`	组件间上下文传递
`@lit-labs/signals`	响应式信号状态管理
`signal-polyfill`	TC39 Signals 提案 Polyfill
`dompurify`	HTML 净化（防 XSS）
`marked`	Markdown 渲染

2.4.5 媒体处理

Table 9: 媒体处理依赖

包名	用途
`sharp`	高性能图像处理
`pdfjs-dist`	PDF 内容提取
`playwright-core`	浏览器自动化
`tar / jszip`	归档与压缩处理

2.5 代码质量工具链

Table 10: 代码质量工具

工具	配置文件	用途
Oxlint	`.oxlintrc.json`	JavaScript/TypeScript 代码检查
Oxfmt	`.oxfmtrc.jsonc`	代码格式化
Shellcheck	—	Shell 脚本检查
SwiftFormat	`.swiftformat`	Swift 代码格式化
SwiftLint	`.swiftlint.yml`	Swift 代码规范
jscpd	—	重复代码检测
knip	`knip.config.ts`	死代码检测
detect-secrets	`.detect-secrets.cfg`	敏感信息检测
zizmor	`zizmor.yml`	GitHub Actions 安全检查
pre-commit	`.pre-commit-config.yaml`	Git 提交前 Hook

3 Monorepo 目录结构总览

3.1 顶层目录

OpenClaw 采用 pnpm workspace 管理的 Monorepo 结构。以下是顶层目录的完整说明：

Table 11: 顶层目录结构详细说明

目录/文件	说明
`src/`	核心源码，包含 77 个子模块，覆盖网关、渠道、智能体、插件、配置、安全、媒体、浏览器自动化、定时任务等全部核心功能
`apps/`	原生应用源码。包含 `macos/`（Swift/SwiftUI 菜单栏应用）、`ios/`（iOS 应用）、`android/`（Kotlin 应用）、`shared/`（OpenClawKit 共享框架）
`extensions/`	42 个官方扩展插件。包括消息渠道集成（Discord、Telegram、Slack 等）和功能扩展（语音通话、LLM 任务、诊断等）
`skills/`	53 个内置技能。提供与第三方服务的集成（GitHub、Notion、1Password 等），通过 ClawHub 分发
`ui/`	基于 Lit Web Components 的控制面板 Web UI，包含 Canvas 宿主。使用 Vite 构建
`docs/`	Mintlify 驱动的文档站点源码，含 47 个子目录。支持中文翻译（`.i18n/` 翻译记忆库）
`scripts/`	114+ 个构建、测试、发布、同步相关的 Shell/JS 脚本
`test/`	端到端测试和测试夹具
`test-fixtures/`	测试用的静态数据和 Mock 文件
`packages/`	共享 npm 包（工作空间内部依赖）
`package.json`	项目清单（v2026.3.9），定义 ESM 模块、CLI 入口和 40+ 导出路径
`pnpm-workspace.yaml`	工作空间定义
`pnpm-lock.yaml`	依赖锁定文件
`tsconfig.json`	TypeScript 严格模式编译配置
`tsdown.config.ts`	构建打包配置
`knip.config.ts`	死代码检测配置
`pyproject.toml`	Python 技能的 Ruff/Pytest 配置
`Dockerfile`	多阶段 Docker 构建（Node 22 bookworm）
`Dockerfile.sandbox`	工具执行沙箱镜像
`docker-compose.yml`	网关 + CLI 服务编排
`docker-setup.sh`	Docker 自动化安装脚本
`setup-podman.sh`	Podman 替代方案安装脚本
`fly.toml`	Fly.io 部署配置
`.oxlintrc.json`	Oxlint 代码检查配置
`.oxfmtrc.jsonc`	Oxfmt 格式化配置
`.swiftlint.yml`	SwiftLint 配置
`.pre-commit-config.yaml`	pre-commit Hook 配置
`.detect-secrets.cfg`	敏感信息检测配置
`.env.example`	环境变量模板
`vitest.config.ts`	主测试配置
`vitest.unit.config.ts`	单元测试配置
`vitest.gateway.config.ts`	网关测试配置
`vitest.e2e.config.ts`	端到端测试配置

目录/文件	说明
vitest.live.config.ts	实时 API 测试配置
vitest.channels.config.ts	渠道测试配置
vitest.extensions.config.ts	扩展测试配置
.github/workflows/	GitHub Actions CI/CD 流水线
CONTRIBUTING.md	贡献指南
SECURITY.md	安全策略
VISION.md	项目愿景
README.md	项目说明

3.2 Workspace 配置

```yaml
packages:
  - '.'              # 根包（核心）
  - 'ui/'            # Web UI 包
  - 'packages/*'     # 共享包
  - 'extensions/*'   # 扩展插件（每个独立包）
```

Listing 2: pnpm-workspace.yaml

这种配置允许每个扩展和 UI 包拥有独立的 package.json 和依赖关系，同时共享根包的公共依赖。

3.3 src/ 目录子模块分类

src/ 包含 77 个子目录，按功能可归类为以下几大类：

3.3.1 CLI 与命令

- cli/ — CLI 基础设施（argparse、profiles、program）
- commands/ — 命令实现（agent、gateway、channels、config 等）
- entry.ts — 程序主入口
- index.ts — 公共 API 导出

3.3.2 网关与通信

- gateway/ — WebSocket 网关服务器（238 源文件，~38K 行核心实现）
- channels/ — 渠道抽象层
- routing/ — 消息路由逻辑
- telegram/, discord/, slack/, signal/, imessage/, line/, web/ — 核心渠道实现

3.3.3 智能体与 AI

- agents/ — Agent 编排（538 源文件，最大子系统）
- auto-reply/ — 自动回复与模板
- context-engine/ — 上下文管理
- providers/ — LLM 模型提供商（13 种）
- acp/ — Agent Control Protocol（31 源文件）

3.3.4 基础设施

- `infra/` — 核心基础设施（300 源文件）：环境变量、二进制管理、设备身份、端口管理、归档处理、Bonjour 发现、路径边界、TLS 指纹
- `config/` — 配置管理（207 源文件）
- `logging/` — 结构化日志（31 源文件）
- `process/` — 进程管理（17 源文件）
- `daemon/` — 守护进程管理（49 源文件）

3.3.5 插件与扩展

- `plugins/` — 插件系统（68 源文件）
- `plugin-sdk/` — 公共插件 API（113 源文件）
- `hooks/` — Hook 系统（38 源文件）

3.3.6 数据与记忆

- `memory/` — 向量记忆系统
- `sessions/` — 会话存储
- `secrets/` — 凭据管理（47 源文件）
- `security/` — 安全特性（31 源文件）

3.3.7 媒体与理解

- `media/` — 媒体处理（42 源文件）
- `media-understanding/` — ML 媒体理解（40 源文件）
- `markdown/` — Markdown 处理（16 源文件）
- `link-understanding/` — 链接提取与理解
- `tts/` — 文本转语音

3.3.8 高级功能

- `browser/` — Playwright 浏览器自动化（135 源文件）
- `cron/` — 定时任务调度（72 源文件）
- `canvas-host/` — Canvas 渲染（8 源文件）
- `node-host/` — Node.js 宿主（18 源文件）
- `wizard/` — 初始化向导
- `terminal/` — 终端 UI（21 源文件）
- `tui/` — 交互式 TUI（33 源文件）

3.3.9 通用支持

- `types/` — 类型定义（11 源文件）
- `shared/` — 共享工具（44 源文件）
- `utils/` — 通用工具函数
- `i18n/` — 国际化（6 语言）
- `compat/` — 兼容性层
- `test-utils/`, `test-helpers/` — 测试工具
- `pairing/` — 设备配对（11 源文件）

3.4 package.json 导出映射

`package.json` 定义了 40+ 个模块导出路径，支持外部工具和插件通过这些路径引用 OpenClaw 的内部模块：

```
{
  "exports": {
    ".": "dist/index.js",
    "./plugin-sdk": "dist/plugin-sdk/index.js",
    "./plugin-sdk/core": "dist/plugin-sdk/core.js",
    "./plugin-sdk/telegram": "dist/plugin-sdk/telegram.js",
    "./plugin-sdk/discord": "dist/plugin-sdk/discord.js",
    "./plugin-sdk/slack": "dist/plugin-sdk/slack.js",
    "./plugin-sdk/signal": "dist/plugin-sdk/signal.js",
    "./plugin-sdk/memory": "dist/plugin-sdk/memory.js",
    "./plugin-sdk/tools": "dist/plugin-sdk/tools.js",
    "./plugin-sdk/acp": "dist/plugin-sdk/acp.js",
    "./plugin-sdk/test-utils": "dist/plugin-sdk/test-utils.js"
  }
}
```

Listing 3: package.json exports 示例（部分）

每个导出路径对应一个独立的编译产物，支持 Tree-shaking，插件开发者只需引入所需的子模块。

4 核心架构设计与数据流

4.1 网关中心型架构

OpenClaw 采用网关中心型（**Gateway-Centric**）架构，WebSocket 网关作为系统的核心中枢，所有消息和控制指令均通过网关进行路由和分发。

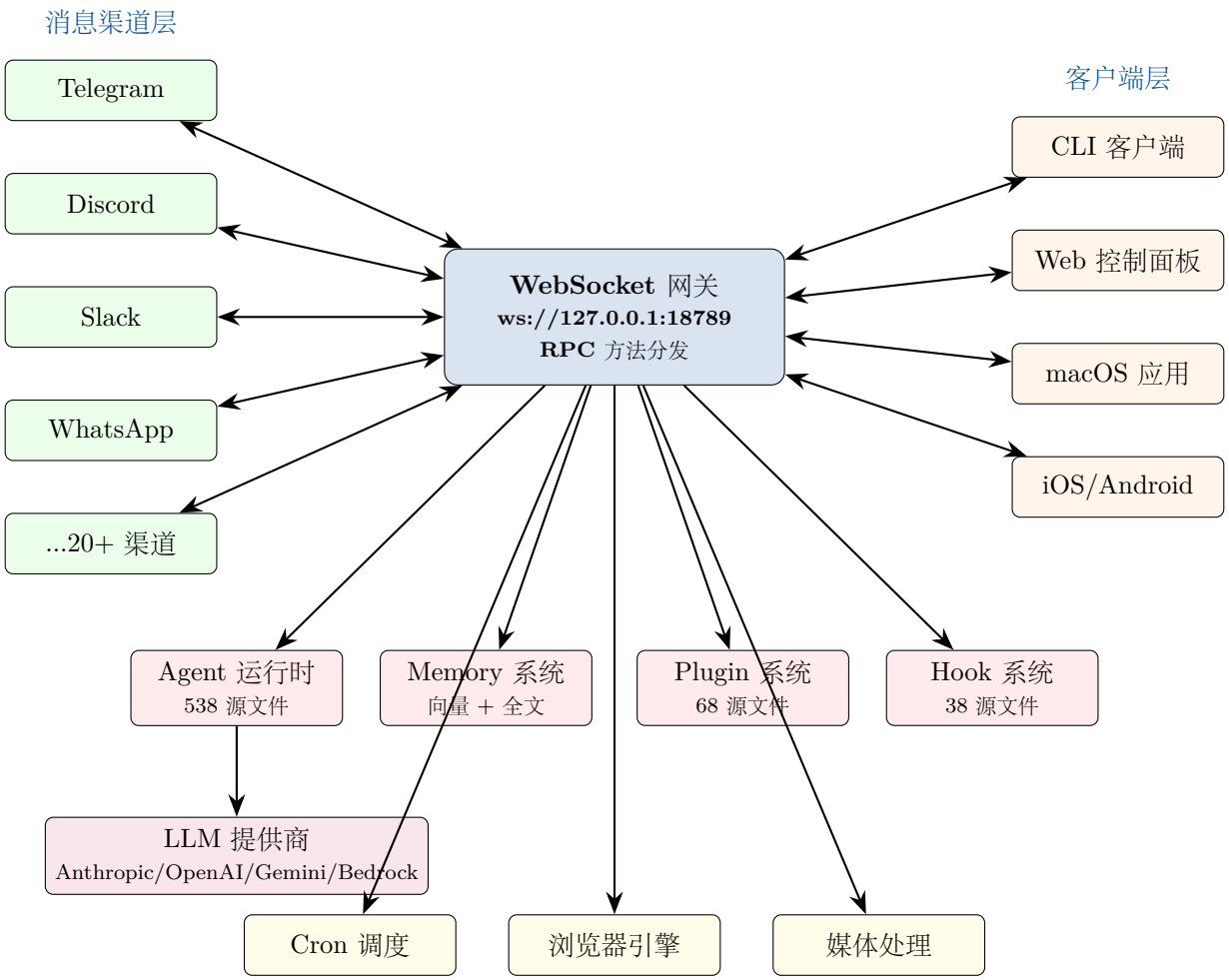

Figure 1: OpenClaw 网关中心型架构全景图

4.2 分层架构

系统从上到下可分为五个逻辑层次：

Table 12: OpenClaw 五层架构

层次	模块	职责
传输层	`ws`, `express`	WebSocket/HTTP 连接管理、TLS
协议层	`gateway/protocol/`	帧格式定义、AJV 验证、版本跟踪
路由层	`routing/`, `channels/`	会话键解析、渠道适配、消息分发
业务层	`agents/`, `plugins/`, `hooks/`	Agent 编排、插件执行、Hook 触发
存储层	`sessions/`, `memory/`, `config/`	会话持久化、向量记忆、配置管理

4.3 消息完整数据流

以下是一条消息从用户发送到 AI 回复的完整处理流程：

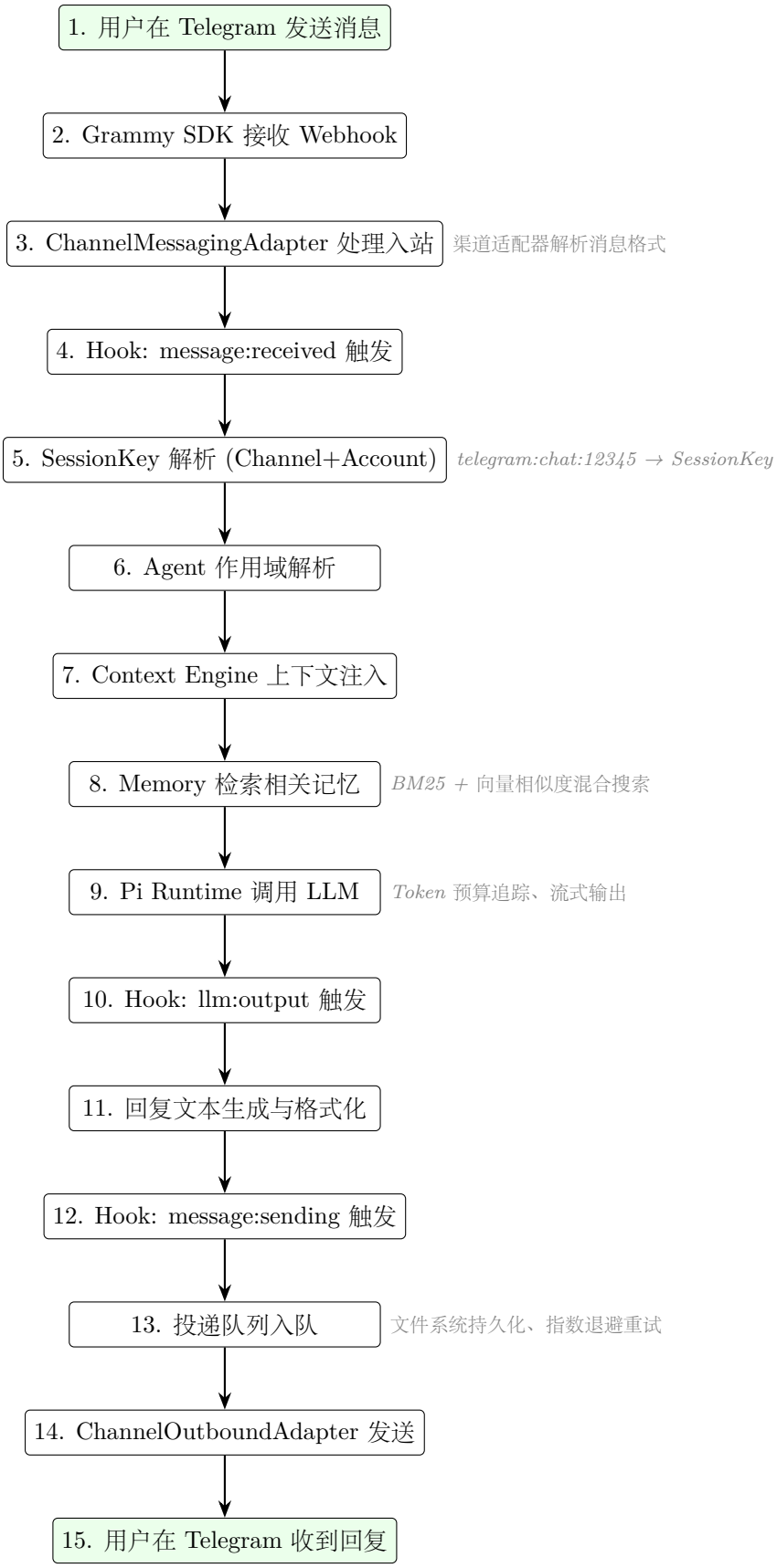

Figure 2: 消息处理全流程（15 步）

4.4 五大设计原则

1. 插件优先（**Plugin-First**）

所有渠道集成、工具和扩展功能均通过统一的插件系统实现。核心代码保持精简，功能扩展通过 8 种注册类型（工具、CLI、HTTP 路由、渠道、提供商、Hook、服务、命令）实现。

2. 适配器模式（**Adapter Pattern**）

   每个渠道的功能被拆分为 10+ 个独立适配器（Config、Security、Outbound、Messaging、Status、Gateway、Pairing、Auth、Command、Action、Setup），每个适配器专注于单一职责，实现了极高的关注点分离度。

3. 事件驱动（**Event-Driven**）

   Hook 系统定义了 17+ 种生命周期事件，覆盖从网关启动到 LLM 调用的全流程。Void Hook 并行执行提高吞吐量，Modifying Hook 串行执行保证一致性。

4. 持久化队列（**Persistent Queue**）

   消息投递采用文件系统持久化队列，使用 temp file → atomic rename → unlink 的三阶段原子操作，配合指数退避重试（5s→25s→2m→10m），即使进程崩溃也不会丢失消息。

5. 角色访问控制（**RBAC**）

   网关方法按角色（operator/node）和权限作用域进行三层授权检查，配合控制面写入预算机制，防止恶意或异常操作。

4.5 关键子系统交互

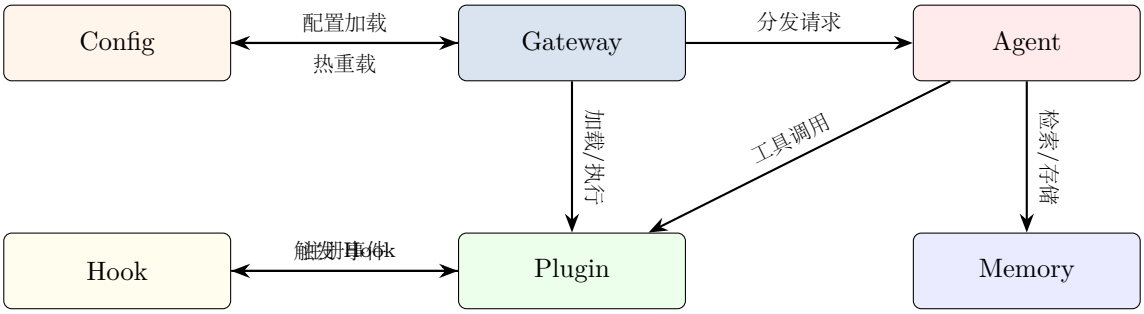

Figure 3: 关键子系统交互关系

5 程序入口与启动流程

5.1 入口文件：entry.ts

src/entry.ts 是整个应用的最外层入口点，由 openclaw.mjs CLI 包装器调用。其核心职责包括：

```
// 1. 防止二次执行（打包器导入保护）
if (globalThis.__OPENCLAW_ENTRY_GUARD__) return;
globalThis.__OPENCLAW_ENTRY_GUARD__ = true;

// 2. 快速路径：--version 和 --help
if (process.argv.includes("--version")) {
  console.log(version);
  process.exit(0);
}

// 3. 进程重生：抑制 ExperimentalWarning
// 如果未设置 NODE_DISABLE_COMPILE_CACHE，
// 重新 fork 进程并添加 --disable-warning 标志
if (shouldRespawn()) {
  respawnWithFlags();
  return;
}

// 4. 解析 CLI Profile
const profile = parseCliProfile(process.argv);

// 5. 委托给 Commander 程序
await program.parseAsync(process.argv);
```

Listing 4: entry.ts 核心逻辑

5.2 模块导出：index.ts

src/index.ts 是模块的公共 API 入口，负责环境初始化：

```
// 1. 静默加载 .env 文件
import "dotenv/config";

// 2. 环境规范化
normalizeEnvironment();

// 3. 确保 openclaw CLI 在 PATH 中
ensureCliBinOnPath();

// 4. 安装全局异常处理器
process.on("unhandledRejection", handleUnhandledRejection);
process.on("uncaughtException", handleUncaughtException);

// 5. 导出公共 API
export { loadConfig, runExec, program, version };
```

Listing 5: index.ts 初始化流程

5.3 CLI 命令树：cli/program.ts

基于 Commander.js 构建完整的命令行界面：

```
openclaw
  |-- gateway              // 启动网关
  |   |-- start            // 启动网关守护进程
  |   |-- stop             // 停止网关
  |   |-- restart          // 重启网关
  |   |-- status           // 查看状态
  |   |-- logs             // 查看日志
  |
  |-- agent                // Agent 管理
  |   |-- list             // 列出 Agent
  |   |-- create           // 创建 Agent
  |   |-- delete           // 删除 Agent
```

```
  |
  |-- channels              // 渠道管理
  |   |-- list              // 列出渠道
  |   |-- status            // 渠道状态
  |
  |-- config                // 配置管理
  |   |-- show              // 显示配置
  |   |-- edit              // 编辑配置
  |   |-- validate          // 验证配置
  |
  |-- secrets               // 凭据管理
  |   |-- audit             // 审计凭据
  |   |-- configure         // 配置凭据
  |
  |-- cron                  // 定时任务
  |-- plugins               // 插件管理
  |-- doctor                // 诊断检查
  |-- onboarding            // 初始设置向导
```

Listing 6: 命令树结构

5.4 网关启动时序

当用户执行 `openclaw gateway start` 时，系统按以下时序完成初始化：

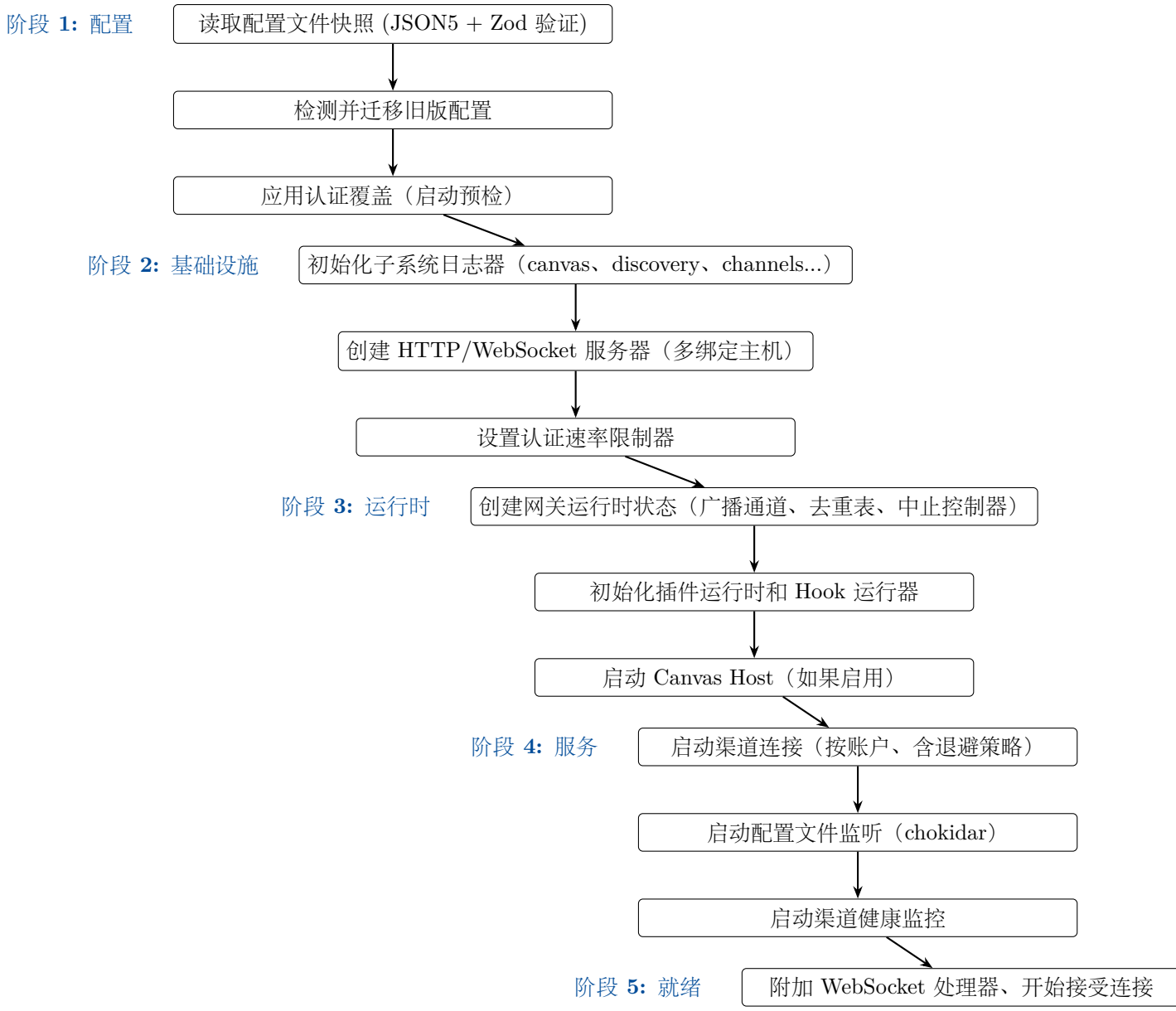

Figure 4: 网关启动时序（5 阶段 13 步）

5.5 依赖注入模式

OpenClaw 采用工厂函数进行依赖注入，而非传统的 DI 容器：

```
type CliDeps = {
  config: OpenClawConfig;
  gateway?: GatewayClient;
  logger: SubsystemLogger;
  secrets: SecretStore;
  // ... 更多依赖
};

// 工厂函数创建默认依赖集
function createDefaultDeps(opts: CreateDepsOptions): CliDeps {
  return {
    config: loadConfig(),
    logger: createSubsystemLogger("cli"),
    secrets: createSecretStore(opts.configDir),
    // ...
  };
}
```

Listing 7: 依赖注入工厂

这种模式的优势在于：

- 类型安全：所有依赖在编译时检查
- 易于测试：测试中可以替换任意依赖
- 无框架开销：不依赖反射或装饰器

6 WebSocket 网关服务器

6.1 核心实现概述

网关服务器的主实现位于 `src/gateway/server.impl.ts`，是整个项目中最大的单一源文件，约 **38,000** 行代码。其核心函数 `startGatewayServer(port, opts)` 负责整个网关的初始化与生命周期管理。

6.2 运行时状态结构

网关运行时状态在 `server-runtime-state.ts` 中创建，包含以下关键组件：

```
type GatewayRuntimeState = {
  // 基础设施
  canvasHost: CanvasHostHandler | null;
  httpServer: HttpServer;
  httpServers: HttpServer[];          // 多绑定主机
  httpBindHosts: string[];
  wss: WebSocketServer;

  // 客户端管理
  clients: Set<GatewayWsClient>;
  broadcast: GatewayBroadcastFn;
  broadcastToConnIds: GatewayBroadcastToConnIdsFn;

  // 聊天运行状态
  agentRunSeq: Map<string, number>;  // Agent 运行序号
  dedupe: Map<string, DedupeEntry>;  // 去重表
  chatRunState: ChatRunState;
  chatRunBuffers: Map<string, string>;
  chatDeltaSentAt: Map<string, number>;
  chatAbortControllers: Map<string, ChatAbortControllerEntry>;

  // 工具事件
  toolEventRecipients: ToolEventRecipientRegistry;
};
```

Listing 8: 网关运行时状态

6.3 WebSocket 连接管理

6.3.1 连接附加

`server-ws-runtime.ts` 中的 `attachGatewayWsHandlers()` 将 WebSocket 事件处理器绑定到服务器：

```
function attachGatewayWsHandlers(params: GatewayWsRuntimeParams) {
  attachGatewayWsConnectionHandler({
    wss,                          // WebSocket 服务器实例
    clients,                      // 活跃客户端集合
    port,                         // 监听端口
    gatewayHost,                  // 网关主机名
    canvasHostEnabled,            // Canvas 功能开关
    canvasHostServerPort,
    resolvedAuth,                 // 认证配置
    rateLimiter,                  // IP 速率限制器
    browserRateLimiter,           // 浏览器专用限制器
    gatewayMethods,               // 所有 RPC 方法
    events,                       // 事件总线
    logGateway,                   // 网关日志器
    logHealth,                    // 健康日志器
    logWsControl,                 // WS 控制日志器
    extraHandlers,                // 插件额外处理器
    broadcast,                    // 广播函数
    buildRequestContext: () => params.context,
  });
}
```

Listing 9: WebSocket 处理器附加

6.3.2 消息处理器

`server/ws-connection/message-handler.ts` 处理所有入站 WebSocket 消息：

1. 帧验证：使用 AJV 验证器校验消息格式
2. 首次连接处理：第一条消息必须是 `connect` 握手
3. 设备签名验证：Ed25519 签名验证（v2/v3 载荷格式）
4. 时间戳偏差检查：容忍 ±2 分钟的时钟偏差
5. 浏览器来源检查：验证 Origin 头部
6. **IP** 速率限制：回环地址有特殊处理
7. 设备元数据更新：更新配对状态和设备信息
8. 请求分发：将验证后的请求转发给对应的 RPC 方法处理器

6.4 广播机制

网关支持两种广播模式：

```
// 1. 全客户端广播
type GatewayBroadcastFn = (
  event: string,
  payload: unknown,
  filter?: (client: GatewayWsClient) => boolean,
) => void;

// 2. 指定连接 ID 广播
type GatewayBroadcastToConnIdsFn = (
  connIds: Set<string>,
  event: string,
  payload: unknown,
) => void;
```

Listing 10: 广播机制

广播用于将状态变更通知所有已连接的客户端，例如渠道状态变更、配置更新、新消息等。

6.5 多绑定主机

网关支持在多个网络接口上监听，提供灵活的网络访问控制：

- 默认绑定 `127.0.0.1`（仅本地访问）
- 可配置绑定 `0.0.0.0`（所有接口）
- 支持多个绑定主机的回退机制
- Tailscale 集成支持远程访问

6.6 Canvas Host 集成

如果启用了 Canvas 功能，网关会在初始化时创建 Canvas Host：

- 根目录验证：确保 Canvas 工作目录存在且安全
- 独立端口：Canvas Host 运行在单独的 HTTP 端口
- 能力 Token：为每个连接生成带 TTL 的 Canvas 能力令牌
- 与 macOS/iOS 原生应用的 A2UI 集成

6.7 优雅关闭

网关关闭时按以下顺序执行清理:

1. 触发 `gateway:stop` Hook
2. 停止接受新连接
3. 中止所有活跃的聊天运行
4. 关闭所有渠道连接
5. 停止配置文件监听
6. 关闭 WebSocket 服务器
7. 关闭 HTTP 服务器
8. 清理 Canvas Host
9. 释放端口和资源

7 网关协议与帧结构

7.1 协议概述

OpenClaw 网关使用自定义的 WebSocket RPC 协议进行客户端-服务器通信。协议定义在 `src/gateway/protocol` 中，使用 AJV（Another JSON Validator）进行运行时 Schema 验证。

7.2 帧类型

协议定义了三种基本帧类型：

Table 13: 协议帧类型

帧类型	方向	用途
RequestFrame	客户端 → 服务器	RPC 方法调用请求
ResponseFrame	服务器 → 客户端	RPC 方法调用响应
EventFrame	服务器 → 客户端	服务器推送的异步事件

7.3 帧结构定义

```
// 请求帧
type RequestFrame = {
  id: string;           // 请求唯一标识
  method: string;       // RPC 方法名
  params: unknown;      // 方法参数
};

// 响应帧
type ResponseFrame = {
  id: string;           // 对应请求的 ID
  result?: unknown;     // 成功时的返回值
  error?: ErrorShape;   // 失败时的错误信息
};

// 事件帧
type EventFrame = {
  type: string;         // 事件类型
  payload: unknown;     // 事件载荷
};

// 错误格式
type ErrorShape = {
  code: string;         // 错误代码
  message: string;      // 人类可读的错误信息
  details?: unknown;    // 额外的调试信息
};
```

Listing 11: 协议帧类型定义

7.4 AJV 验证器

协议为每种帧类型和关键参数定义了严格的 JSON Schema 验证器：

```
import Ajv from "ajv";

const ajv = new Ajv({ strict: false });

// 导出的验证器
export const validateConnectParams = ajv.compile(connectParamsSchema);
export const validateRequestFrame = ajv.compile(requestFrameSchema);
export const validateResponseFrame = ajv.compile(responseFrameSchema);
export const validateEventFrame = ajv.compile(eventFrameSchema);
export const validateSendParams = ajv.compile(sendParamsSchema);
export const validateChatSendParams = ajv.compile(chatSendParamsSchema);
```

Listing 12: AJV 验证器配置

`strict: false` 模式允许额外属性，保证协议的前向兼容性——旧版服务器可以处理新版客户端发送的包含未知字段的消息。

7.5 ConnectParams 握手参数

连接握手是每个 WebSocket 会话的第一步，`ConnectParams` 包含：

```
type ConnectParams = {
  // 设备身份
  deviceId: string;
  clientId: string;
  clientMode: "cli" | "web" | "node" | "webchat";
  platform?: string;            // "macos", "ios", "android", ...
  deviceFamily?: string;

  // 认证
  role?: "operator" | "node";  // 默认 operator
  scopes?: string[];            // 权限作用域列表
  token?: string;               // 认证令牌

  // 签名验证（v3）
  signedAtMs?: number;          // 签名时间戳
  nonce?: string;               // 一次性随机数
  signature?: string;           // Ed25519 签名

  // 协议
  protocolVersion?: number;     // 协议版本号
};
```

Listing 13: 连接握手参数

7.6 ChatSendParams 聊天发送参数

```
type ChatSendParams = {
  text: string;                     // 消息文本
  sessionKey?: string;              // 目标会话
  agentId?: string;                 // 目标 Agent
  attachments?: Attachment[];       // 附件列表

  // 路由信息
  originatingChannel?: string;      // 来源渠道
  originatingTo?: string;           // 来源目标
  accountId?: string;               // 多账户支持
  messageThreadId?: string | number; // 线程 ID
  explicitDeliverRoute?: boolean;   // 显式路由

  // 选项
  thinkingLevel?: string;           // 思考深度
  modelOverride?: string;           // 模型覆盖
};
```

Listing 14: 聊天发送参数

7.7 协议版本管理

协议维护版本号以确保客户端和服务器之间的兼容性：

- 客户端在连接时报告其支持的协议版本
- 服务器根据版本号决定是否使用新特性
- 低版本客户端可以连接高版本服务器（向后兼容）
- 不兼容的版本变更会在连接时返回错误

8 RPC 方法分发与处理器

8.1 方法聚合器

`server-methods.ts` 将所有领域处理器聚合为一个统一的处理器表：

```
export const coreGatewayHandlers: GatewayRequestHandlers = {
  ...connectHandlers,          // 连接与握手
  ...logsHandlers,             // 日志查询
  ...voicewakeHandlers,        // 语音唤醒
  ...healthHandlers,           // 健康检查
  ...channelsHandlers,         // 渠道操作
  ...chatHandlers,             // 聊天处理
  ...configHandlers,           // 配置管理
  ...configPatchHandlers,      // 配置补丁
  ...cronHandlers,             // 定时任务
  ...sessionHandlers,          // 会话管理
  ...agentHandlers,            // 单 Agent 操作
  ...agentsHandlers,           // 多 Agent 操作
  ...deviceHandlers,           // 设备管理
  ...devicePairingHandlers,    // 设备配对
  ...doctorHandlers,           // 诊断检查
  ...execApprovalsHandlers,    // 执行审批
  ...webHandlers,              // Web 相关
  ...modelsHandlers,           // 模型目录
  ...nodeHandlers,             // 移动节点
  ...pushHandlers,             // 推送通知
  ...sendHandlers,             // 消息发送
  ...usageHandlers,            // 使用统计
  ...browserHandlers,          // 浏览器自动化
  ...systemHandlers,           // 系统操作
  // ... 共计 30+ 处理器组
};
```

Listing 15: 核心 RPC 方法聚合

8.2 请求处理完整流程

`handleGatewayRequest()` 函数实现了完整的请求处理管线：

```
export async function handleGatewayRequest(
  opts: GatewayRequestOptions & {
    extraHandlers?: GatewayRequestHandlers;
  },
): Promise<void> {
  const { req, client, respond, context } = opts;
  const { method } = req;

  // 步骤 1: 角色授权检查
  const authError = authorizeGatewayMethod(method, client);
  if (authError) {
    return respond(false, null, authError);
  }

  // 步骤 2: 控制面写入预算检查
  if (isControlPlaneWriteMethod(method)) {
    const budget = consumeControlPlaneWriteBudget({ client });
    if (!budget.allowed) {
      return respond(false, null, "rate_limited");
    }
  }

  // 步骤 3: 查找处理器（优先插件扩展）
  const handler = opts.extraHandlers?.[method]
    ?? coreGatewayHandlers[method];

  if (!handler) {
    return respond(false, null, {
      code: "METHOD_NOT_FOUND",
      message: `Unknown method: ${method}`,
    });
  }
```

```
  // 步骤 4: 在插件运行时作用域内执行
  await withPluginRuntimeGatewayRequestScope(
    { context, isWebchatConnect: false },
    async () => {
      try {
        await handler({
          method, params: req.params,
          req, client, respond, context,
        });
      } catch (error) {
        respond(false, null, formatError(error));
      }
    },
  );
}
```

Listing 16: 请求处理管线

8.3 请求上下文

每个 RPC 方法处理器都接收一个丰富的请求上下文:

```
type GatewayRequestContext = {
  // 依赖注入
  deps: ReturnType<typeof createDefaultDeps>;

  // 定时任务
  cron: CronService;
  cronStorePath: string;

  // 模型目录
  loadGatewayModelCatalog: () => Promise<ModelCatalogEntry[]>;

  // 健康监控
  getHealthCache: () => HealthSummary | null;
  refreshHealthSnapshot: (opts?) => Promise<HealthSummary>;
  logHealth: SubsystemLogger;
  logGateway: SubsystemLogger;

  // 版本追踪
  incrementPresenceVersion: () => number;
  getHealthVersion: () => number;

  // 广播
  broadcast: GatewayBroadcastFn;
  broadcastToConnIds: GatewayBroadcastToConnIdsFn;

  // 移动节点
  nodeSendToSession: (sessionKey, event, payload) => void;
  nodeRegistry: NodeRegistry;

  // 聊天状态
  agentRunSeq: Map<string, number>;
  chatAbortControllers: Map<string, ChatAbortControllerEntry>;
  chatRunBuffers: Map<string, string>;

  // 去重与向导
  dedupe: Map<string, DedupeEntry>;
  wizardSessions: Map<string, WizardSession>;
};
```

Listing 17: GatewayRequestContext 完整定义

8.4 主要处理器组详解

8.4.1 chat 处理器

Table 14: Chat RPC 方法

方法	功能
`chat.send`	发送消息，触发 Agent 处理，路由到目标渠道
`chat.history`	获取会话消息历史（支持分页）
`chat.inject`	注入消息到转录（测试/恢复用途）
`chat.abort`	中止活跃的聊天运行（支持去重）

8.4.2 chat.send 投递路由

```
type ChatSendOriginatingRoute = {
  originatingChannel: string;      // "internal" 或渠道 ID
  originatingTo?: string;          // 目标标识符
  accountId?: string;              // 多账户支持
  messageThreadId?: string | number;
  explicitDeliverRoute: boolean;   // 是否显式路由
};
```

Listing 18: 聊天发送路由结构

8.4.3 config 处理器

方法	功能
`config.get`	获取当前配置快照
`config.apply`	应用完整配置（触发重载）
`config.patch`	部分配置补丁
`config.validate`	验证配置（不应用）
`config.schema`	获取配置 JSON Schema

8.4.4 其他关键处理器

Table 15: 其他关键 RPC 方法

组	方法示例	功能
channels	`channels.status`	获取所有渠道连接状态
agents	`agents.list`	列出所有 Agent 和其配置
sessions	`sessions.list`	列出活跃会话及使用统计
cron	`cron.list`, `cron.create`	管理定时任务
device	`device.pair`, `device.unpair`	设备配对管理
health	`health.check`	网关健康检查
models	`models.catalog`	获取可用模型列表
usage	`usage.summary`	Token/成本使用统计

8.5 插件方法扩展

插件可以通过 `PluginGatewayAdapter` 注册额外的 RPC 方法：

```
// 插件注册的额外处理器通过 extraHandlers 参数传入
const handler = opts.extraHandlers?.[method]
  ?? coreGatewayHandlers[method];
// 插件处理器优先于核心处理器
```

Listing 19: 插件注册自定义 RPC 方法

这使得插件可以:

- 添加全新的 RPC 方法
- 覆盖现有方法的行为
- 在不修改核心代码的情况下扩展网关功能

9 认证、授权与 RBAC

9.1 认证体系概述

OpenClaw 的认证体系分为三个层次：

1. 传输层认证：TLS 指纹验证和设备签名
2. 连接层认证：WebSocket 握手时的 Token 和角色验证
3. 方法层授权：每个 RPC 方法调用的权限检查

9.2 设备签名验证

9.2.1 Ed25519 签名

设备连接使用 Ed25519 非对称密钥进行签名验证，支持 v2 和 v3 两种载荷格式：

```
type DeviceAuthPayloadV3 = {
  deviceId: string;          // 设备唯一标识
  clientId: string;          // 客户端实例标识
  clientMode: string;        // cli | web | node | webchat
  role: string;              // operator | node
  scopes: string[];          // 权限作用域
  signedAtMs: number;        // 签名时间戳（毫秒）
  nonce: string;             // 一次性随机数（防重放）
  token?: string;            // 认证令牌
  platform?: string;         // 平台标识
  deviceFamily?: string;     // 设备家族
};
// 使用 Ed25519 私钥对序列化载荷进行签名
```

Listing 20: 设备认证载荷 v3

9.2.2 时间戳偏差容忍

签名验证包含时间戳检查，容忍 ±2 分钟的时钟偏差：

```
const TIMESTAMP_SKEW_TOLERANCE_MS = 2 * 60 * 1000; // 2 分钟

function isTimestampValid(signedAtMs: number): boolean {
  const now = Date.now();
  const diff = Math.abs(now - signedAtMs);
  return diff <= TIMESTAMP_SKEW_TOLERANCE_MS;
}
```

Listing 21: 时间戳偏差检查

9.3 角色与权限模型

9.3.1 角色定义

Table 16: 网关角色

角色	适用场景	权限范围
`operator`	人类用户（CLI、Web UI、桌面应用）	完全访问，可通过 scopes 限制
`node`	移动节点（iOS、Android）	受限访问，仅限节点相关操作

9.3.2 三层授权检查

`role-policy.ts` 实现了三层渐进式授权：

```
function authorizeGatewayMethod(
  method: string,
  client: GatewayClient,
): AuthError | null {

  // ===== 第一层：角色检查 =====
  const role = parseGatewayRole(client.connect.role ?? "operator");
  if (!isRoleAuthorizedForMethod(role, method)) {
    return {
      code: "UNAUTHORIZED",
      message: `Role '${role}' cannot access '${method}'`,
    };
  }

  // ===== 第二层：作用域检查（限制型 operator）=====
  const scopes = client.connect.scopes ?? [];
  if (scopes.length > 0) {
    // 非空 scopes 表示受限 operator
    const scopeAuth = authorizeOperatorScopesForMethod(method, scopes);
    if (!scopeAuth.allowed) {
      return {
        code: "MISSING_SCOPE",
        message: `Scope '${scopeAuth.requiredScope}' needed`,
      };
    }
  }

  // ===== 第三层：控制面写入预算 =====
  if (CONTROL_PLANE_WRITE_METHODS.includes(method)) {
    const budget = consumeControlPlaneWriteBudget(client);
    if (!budget.allowed) {
      return {
        code: "RATE_LIMITED",
        message: "Control plane write budget exceeded",
      };
    }
  }

  return null; // 授权通过
}
```

Listing 22: 三层授权模型

9.4 速率限制

9.4.1 IP 速率限制

连接握手阶段对每个 IP 地址进行速率限制，防止暴力破解：

- 普通 IP：严格的连接频率限制
- 回环地址（127.0.0.1）：放宽限制（本地开发场景）
- 浏览器专用限制器：针对 Web UI 的额外限制

9.4.2 控制面写入预算

`control-plane-rate-limit.ts` 实现了控制面写入操作的预算机制：

```
// 控制面写入方法列表
const CONTROL_PLANE_WRITE_METHODS = [
  "config.apply",
  "config.patch",
  "update.run",
  // ... 其他写入方法
];

// 每个客户端维护独立的写入预算
function consumeControlPlaneWriteBudget(
  client: GatewayClient,
```

```
): { allowed: boolean } {
  // 令牌桶算法：每时间窗口恢复固定数量的令牌
  // 每次写入操作消耗一个令牌
  // 令牌耗尽时拒绝写入
}
```

Listing 23: 写入预算机制

9.5 静默本地配对

为了简化本地使用体验，系统在以下条件下自动批准设备配对：

- 连接来自回环地址（本地客户端）
- 非浏览器来源（CLI 工具）
- Control UI 或 Webchat 客户端模式
- 首次配对或权限升级场景

9.6 浏览器来源验证

Web UI 连接需要通过额外的来源验证：

```
function validateBrowserOrigin(
  origin: string | undefined,
  request: IncomingMessage,
): boolean {
  // 1. 检查 Origin 头
  // 2. 如果无 Origin，检查 Sec-WebSocket-Origin
  // 3. 验证来源是否在允许列表中
  // 4. 本地回环地址来源自动允许
}
```

Listing 24: 浏览器来源检查

10 渠道集成架构

10.1 ChannelPlugin 泛型类型

每个渠道通过实现 `ChannelPlugin` 泛型接口进行集成。该接口使用三个类型参数来适配不同渠道的数据模型：

```
type ChannelPlugin<
  ResolvedAccount,   // 渠道解析后的账户配置
  Probe,             // 健康探测结果类型
  Audit,             // 安全审计结果类型
> = {
  id: ChannelId;                          // 渠道唯一标识
  meta: ChannelMeta;                      // 渠道元数据
  capabilities: ChannelCapabilities;      // 功能矩阵

  // 10+ 适配器组合
  config: ChannelConfigAdapter<ResolvedAccount>;
  security: ChannelSecurityAdapter<ResolvedAccount>;
  outbound: ChannelOutboundAdapter;
  messaging: ChannelMessagingAdapter;
  status: ChannelStatusAdapter<ResolvedAccount, Probe, Audit>;
  gateway: ChannelGatewayAdapter<ResolvedAccount>;
  pairing: ChannelPairingAdapter;
  auth: ChannelAuthAdapter;
  commands: ChannelCommandAdapter;
  actions: ChannelMessageActionAdapter;
  setup: ChannelSetupAdapter;
};
```

Listing 25: ChannelPlugin 泛型定义

10.2 渠道能力矩阵

`ChannelCapabilities` 定义了每个渠道支持的功能集：

```
type ChannelCapabilities = {
  chatTypes: Array<ChatType | "thread">;  // 支持的聊天类型
  polls?: boolean;              // 投票功能
  reactions?: boolean;          // 消息反应
  edit?: boolean;               // 消息编辑
  unsend?: boolean;             // 消息撤回
  reply?: boolean;              // 消息回复引用
  effects?: boolean;            // 消息特效
  groupManagement?: boolean;    // 群组管理
  threads?: boolean;            // 线程/话题
  media?: boolean;              // 媒体文件支持
  nativeCommands?: boolean;     // 原生命令（/command）
  blockStreaming?: boolean;     // 块流式输出
};
```

Listing 26: 渠道能力矩阵

10.3 渠道账户快照

每个渠道账户的运行时状态通过 `ChannelAccountSnapshot` 追踪：

```
type ChannelAccountSnapshot = {
  // 身份
  accountId: string;

  // 生命周期状态
  enabled?: boolean;            // 是否启用
  configured?: boolean;         // 是否已配置
  linked?: boolean;             // 是否已关联
  running?: boolean;            // 是否正在运行
  connected?: boolean;          // 是否已连接

  // 运行统计（20+ 字段）
  lastConnectedAt?: number | null;
```

```
  lastDisconnect?: string | {
    at: number;
    status?: number;
    error?: string;
  };
  activeRuns?: number;          // 活跃运行数

  // 渠道特定数据
  probe?: unknown;              // 健康探测结果
  audit?: unknown;              // 审计信息
};
\end{lstlisting>

\subsection{支持渠道完整列表}

\begin{longtable}{l l l l}
\caption{全部支持的消息渠道} \\
\toprule
\textbf{渠道} & \textbf{位置} & \textbf{类型} & \textbf{SDK/协议} \\
\midrule
\endfirsthead
\toprule
\textbf{渠道} & \textbf{位置} & \textbf{类型} & \textbf{SDK/协议} \\
\midrule
\endhead
Telegram & \texttt{src/telegram/} & 核心 & Grammy \\
Discord & \texttt{src/discord/} & 核心 & discord-api-types \\
Slack & \texttt{src/slack/} & 核心 & @slack/bolt \\
Signal & \texttt{src/signal/} & 核心 & Signal CLI Bridge \\
WhatsApp & \texttt{src/web/} & 核心 & baileys \\
iMessage & \texttt{src/imessage/} & 核心 & AppleScript/BlueBubbles \\
LINE & \texttt{src/line/} & 核心 & @line/bot-sdk \\
\midrule
IRC & \texttt{ext/irc/} & 扩展 & IRC 协议 \\
Matrix & \texttt{ext/matrix/} & 扩展 & Matrix SDK + E2EE \\
MS Teams & \texttt{ext/msteams/} & 扩展 & Bot Framework \\
Google Chat & \texttt{ext/googlechat/} & 扩展 & Workspace API \\
飞书 & \texttt{ext/feishu/} & 扩展 & @larksuiteoapi \\
Mattermost & \texttt{ext/mattermost/} & 扩展 & Mattermost API \\
Twitch & \texttt{ext/twitch/} & 扩展 & Twitch IRC \\
Nostr & \texttt{ext/nostr/} & 扩展 & Nostr 协议 \\
Nextcloud Talk & \texttt{ext/nextcloud-talk/} & 扩展 & Nextcloud API \\
Synology Chat & \texttt{ext/synology-chat/} & 扩展 & Synology API \\
Tlon & \texttt{ext/tlon/} & 扩展 & Tlon API \\
Zalo & \texttt{ext/zalo/} & 扩展 & Zalo OA API \\
BlueBubbles & \texttt{ext/bluebubbles/} & 扩展 & BlueBubbles API \\
\bottomrule
\end{longtable}

\subsection{核心渠道 vs 扩展渠道}

\begin{table}[H]
\centering
\caption{核心渠道与扩展渠道的区别}
\begin{tabularx}{\textwidth}{l X X}
\toprule
\textbf{维度} & \textbf{核心渠道} & \textbf{扩展渠道} \\
\midrule
位置 & \texttt{src/<channel>/} & \texttt{extensions/<channel>/} \\
依赖 & 打包在主 package 中 & 独立 \texttt{package.json} \\
加载 & 随网关自动加载 & 通过插件系统动态加载 \\
更新 & 随主版本更新 & 可独立更新 \\
测试 & 主测试套件 & 独立测试配置 \\
\bottomrule
\end{tabularx}
\end{table}

\subsection{渠道生命周期管理}

\texttt{src/gateway/server-channels.ts} 负责所有渠道的生命周期:

\begin{lstlisting}[caption={渠道生命周期}]
// 1. 启动: 按账户启动, 支持退避策略
async function startChannel(channelId, accountId) {
  // 检查是否已配置和启用
```

```
  // 应用退避策略（上次失败后等待）
  // 创建渠道运行时上下文
  // 调用 ChannelGatewayAdapter.start()
  // 更新账户状态快照
}

// 2. 重启退避策略
const BACKOFF_SCHEDULE = [
  5_000,      // 首次重试: 5 秒
  15_000,     // 第二次: 15 秒
  30_000,     // 第三次: 30 秒
  60_000,     // 第四次: 1 分钟
  300_000,    // 最终: 5 分钟
];

// 3. 健康快照
function getChannelHealthSnapshot(): ChannelAccountSnapshot[] {
  // 聚合所有渠道账户的状态
  // 返回连接状态、活跃运行数、探测结果等
}
```

Listing 27: 渠道账户运行时状态

11 渠道适配器详解

11.1 适配器组合模式

OpenClaw 采用适配器组合模式替代传统的单体渠道类。每个渠道的功能被拆分为 10+ 个独立适配器，每个适配器专注于单一职责。这种设计使得新渠道只需实现必要的适配器即可接入系统。

11.2 ChannelConfigAdapter — 配置适配器

负责渠道账户的配置管理：

```
type ChannelConfigAdapter<ResolvedAccount> = {
  // 列出所有已配置的账户 ID
  listAccountIds: (cfg: OpenClawConfig) => string[];

  // 解析账户配置为强类型对象
  resolveAccount: (
    cfg: OpenClawConfig,
    accountId?: string,
  ) => ResolvedAccount;

  // 检查账户是否启用
  isEnabled?: (
    account: ResolvedAccount,
    cfg: OpenClawConfig,
  ) => boolean;

  // 检查账户是否已完成配置
  isConfigured?: (
    account: ResolvedAccount,
    cfg: OpenClawConfig,
  ) => boolean | Promise<boolean>;

  // 生成账户状态描述
  describeAccount?: (
    account: ResolvedAccount,
    cfg: OpenClawConfig,
  ) => ChannelAccountSnapshot;
};
```

Listing 28: 配置适配器接口

11.3 ChannelSecurityAdapter — 安全适配器

控制消息接收策略和访问白名单：

```
type ChannelSecurityAdapter<ResolvedAccount> = {
  // DM 策略: pairing (需配对) 或 open (开放+白名单)
  dmPolicy: (account: ResolvedAccount) => "pairing" | "open";

  // 获取白名单条目
  getAllowlist?: (
    account: ResolvedAccount,
    cfg: OpenClawConfig,
  ) => AllowlistEntry[];

  // 验证是否允许接收消息
  isAllowed?: (
    sender: ChannelSender,
    account: ResolvedAccount,
  ) => boolean;

  // 群组白名单 (带角色要求)
  getGroupAllowlist?: (
    account: ResolvedAccount,
  ) => GroupAllowlistEntry[];
};
```

Listing 29: 安全适配器接口

11.4 ChannelOutboundAdapter — 出站适配器

处理消息发送逻辑：

```
type ChannelOutboundAdapter = {
  // 投递模式
  deliveryMode: "direct" | "gateway" | "hybrid";

  // 文本分块限制（如 Telegram 4096 字符）
  textChunkLimit?: number;

  // 自定义分块逻辑
  chunker?: (
    text: string,
    limit: number,
  ) => string[];

  // 解析投递目标
  resolveTarget?: (
    cfg: OpenClawConfig,
    to: string,
    allowFrom: string[],
    accountId: string,
    mode: ChannelOutboundTargetMode,
  ) => { ok: true; to: string } | { ok: false; error: string };

  // 发送载荷（通用）
  sendPayload?: (ctx: OutboundContext) => Promise<OutboundDeliveryResult>;

  // 发送纯文本
  sendText?: (ctx: OutboundContext) => Promise<OutboundDeliveryResult>;
};
```

Listing 30: 出站适配器接口

11.5 ChannelSetupAdapter — 设置适配器

处理渠道的初始配置和账户创建：

```
type ChannelSetupAdapter = {
  // 解析账户 ID
  resolveAccountId?: (
    cfg: OpenClawConfig,
    accountId?: string,
    input?: unknown,
  ) => string;

  // 应用账户配置到全局配置
  applyAccountConfig: (
    cfg: OpenClawConfig,
    accountId: string,
    input: unknown,
  ) => OpenClawConfig;

  // 验证输入参数
  validateInput?: (
    cfg: OpenClawConfig,
    accountId: string,
    input: unknown,
  ) => string | null;  // 返回错误信息或 null
};
```

Listing 31: 设置适配器接口

11.6 其他适配器

Table 17: 其他适配器接口

适配器	职责
ChannelMessagingAdapter	处理入站消息的解析、格式化和分发
ChannelStatusAdapter	健康监控、连接状态探测、审计信息收集
ChannelGatewayAdapter	注册自定义网关方法、管理渠道运行时上下文
ChannelPairingAdapter	设备配对流程（QR 码生成、配对码验证）
ChannelAuthAdapter	OAuth 流程、Token 刷新、凭据存储
ChannelCommandAdapter	注册渠道专属的 CLI 命令
ChannelMessageActionAdapter	消息动作（回复、转发、反应等）

11.7 Telegram 渠道实现深度分析

以 Telegram 为例，展示一个完整渠道的实现细节。其核心实现在 `extensions/telegram/src/channel.ts`（558 行）：

11.7.1 能力声明

```
capabilities: {
  chatTypes: ["dm", "group", "thread"],
  polls: true,
  reactions: true,
  edit: true,
  unsend: true,
  reply: true,
  media: true,
  nativeCommands: true,
  blockStreaming: true,
  threads: true,
},
```

Listing 32: Telegram 能力矩阵

11.7.2 出站配置

```
outbound: {
  deliveryMode: "direct",
  textChunkLimit: 4000,  // Telegram Markdown 限制
  chunker: telegramMarkdownChunker,
  resolveTarget: (cfg, to, allowFrom, accountId) => {
    // 验证目标聊天 ID
    // 支持数字 ID 和 @username 格式
  },
  sendPayload: async (ctx) => {
    // 发送文本、媒体、投票等
    // 支持回复引用和线程
  },
},
```

Listing 33: Telegram 出站适配器

11.7.3 安全策略

```
security: {
  dmPolicy: (account) => account.dmPolicy ?? "pairing",
  getAllowlist: (account) => account.allowlist ?? [],
  getGroupAllowlist: (account) => account.groupAllowlist ?? [],
  isAllowed: (sender, account) => {
    // 检查发送者是否在白名单中
```

```
    // 支持 Telegram 数字 ID 和用户名
  },
},
```

Listing 34: Telegram 安全配置

11.7.4 多账户支持

Telegram 扩展支持多个 Bot Token 同时运行，并防止 Token 在不同账户间重复使用：

```
config: {
  listAccountIds: (cfg) => {
    // 遍历 cfg.telegram.accounts
    // 返回所有已配置的账户 ID
  },
  resolveAccount: (cfg, accountId) => {
    // 解析 Bot Token、Webhook 配置
    // 处理 polling/webhook 模式切换
  },
},
```

Listing 35: Telegram 多账户管理

12 消息路由与会话管理

12.1 会话键（SessionKey）

会话键是 OpenClaw 中消息组织的核心概念，由渠道和账户信息组合生成：

```
// session-key.ts
type SessionKey = string;
// 格式: "<channel>:<scope>:<target>"
// 示例: "telegram:chat:123456789"
//       "discord:guild:987654:channel:111222"
//       "slack:team:T01:channel:C02"

function resolveSessionKey(params: {
  channel: ChannelId;
  chatId: string;
  threadId?: string;
  accountId?: string;
}): SessionKey {
  // 1. 规范化渠道标识
  // 2. 规范化聊天/群组/线程 ID
  // 3. 处理账户隔离（多账户场景）
  // 4. 组合生成唯一会话键
}
```

Listing 36: 会话键解析

12.2 入站路由流程

当一条消息从外部渠道进入系统时，路由过程如下：

1. 渠道适配器接收：各渠道 SDK（Grammy、Bolt 等）接收原始消息
2. 消息规范化：将渠道特定格式转换为统一的内部消息格式
3. 安全检查：通过 SecurityAdapter 验证发送者权限
4. 会话键生成：根据 Channel + Chat ID 生成 SessionKey
5. **Agent** 解析：通过 `resolveSessionAgentIds()` 确定目标 Agent
6. 会话查找/创建：在会话存储中查找或创建对应会话
7. 消息入队：将消息追加到会话转录中
8. **Agent** 触发：唤醒 Agent 运行时处理消息

12.3 出站路由

```
type ChatSendOriginatingRoute = {
  // 消息来源渠道
  originatingChannel: string;  // "internal" 或具体渠道 ID

  // 来源目标标识
  originatingTo?: string;

  // 多账户场景下的账户 ID
  accountId?: string;

  // 消息线程
  messageThreadId?: string | number;

  // 是否为显式路由（用户指定投递目标）
  explicitDeliverRoute: boolean;
};

type OutboundTarget = {
```

```
  channel: OutboundChannel;
  to?: string;
  accountId?: string;
  threadId?: string | number;
  reason?: string;  // 路由决策原因
};
```

Listing 37: 出站路由结构

12.4 会话存储

12.4.1 JSONL 格式

会话消息以 JSONL（换行分隔的 JSON）格式存储：

```
# 存储路径
~/.openclaw/agents/<agentId>/sessions/<sessionKey>.jsonl

# 每行一条消息记录
{"id":"msg_001","parentId":null,"role":"user","text":"Hello"}
{"id":"msg_002","parentId":"msg_001","role":"assistant","text":"Hi!"}
{"id":"msg_003","parentId":"msg_002","role":"user","text":"How are you?"}
```

Listing 38: 会话存储路径和格式

12.4.2 DAG 消息结构

消息通过 `parentId` 形成有向无环图（DAG）结构，而非简单的线性列表：

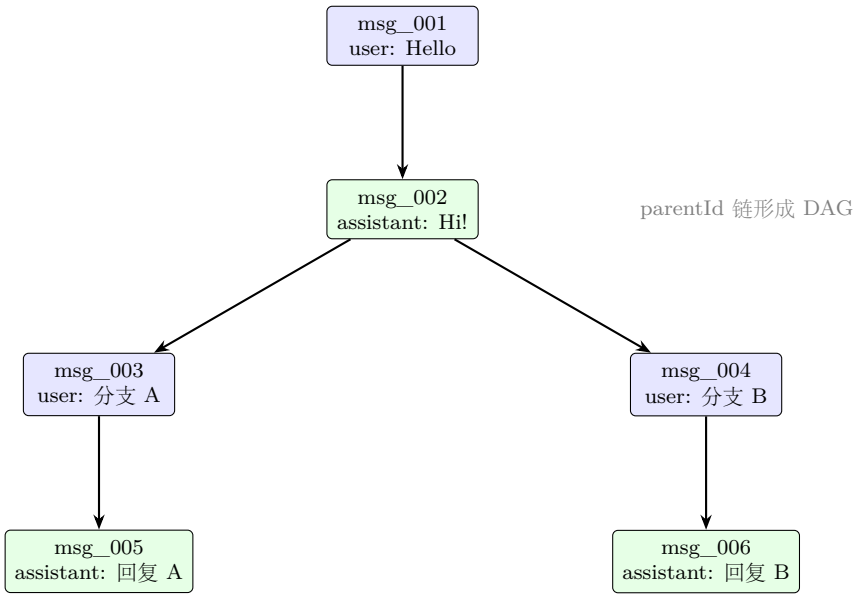

Figure 5: 消息 DAG 结构示例

DAG 结构的优势：

- 支持对话分支和回溯
- 便于会话压缩（compaction）
- 支持线程化展示
- 保持历史完整性

12.5 会话类型

Table 18: 会话模式

模式	说明
`main`	直接一对一对话会话
群组隔离	群聊中为每个群组创建独立会话
队列模式	消息按队列处理，支持优先级和公平调度
激活模式	会话需要通过特定触发器激活（如 @mention）

12.6 session-utils.ts 工具集

`src/gateway/session-utils.ts` 是一个 ~28,000 行的大型工具模块，提供：

- 会话查找：按 SessionKey、渠道、Agent 等条件查找
- 会话过滤：按时间范围、活跃状态等过滤
- 会话补丁：修改会话元数据
- 转录注入：向现有会话中注入消息
- 转录压缩：压缩过长的会话转录
- 使用统计：计算 Token 使用量和成本

12.7 并发控制

12.7.1 KeyedAsyncQueue

```
// plugin-sdk/keyed-async-queue.ts

class KeyedAsyncQueue {
  private queues = new Map<string, Promise<void>>();

  async enqueue(key: string, task: () => Promise<void>) {
    // 获取当前键的队列尾部
    const prev = this.queues.get(key) ?? Promise.resolve();

    // 将新任务链接到队列尾部
    const next = prev.then(task).catch(() => {}).finally(() => {
      if (this.queues.get(key) === next) {
        this.queues.delete(key);
      }
    });

    this.queues.set(key, next);
    return next;
  }
}

// 用途：确保同一 session/thread 内的操作串行执行
// 防止并发修改导致的数据竞争
```

Listing 39: 基于键的异步队列

12.7.2 去重机制

```
function setGatewayDedupeEntry(
  dedupe: Map<string, DedupeEntry>,
  key: string,
  ttlMs: number,
) {
  dedupe.set(key, { expiresAt: Date.now() + ttlMs });
  // 定期清理过期条目
}
```

Listing 40: 去重入口

13 消息投递队列系统

13.1 架构设计

消息投递队列位于 `src/infra/outbound/delivery-queue.ts`，采用文件系统持久化方案，确保消息即使在进程崩溃后也不会丢失。

13.2 QueuedDelivery 完整结构

```typescript
type QueuedDeliveryPayload = {
  // 目标
  channel: Exclude<OutboundChannel, "none">;
  to: string;
  accountId?: string;

  // 消息内容
  payloads: ReplyPayload[];      // 原始消息载荷（多段）

  // 线程与引用
  threadId?: string | number | null;
  replyToId?: string | null;

  // 投递选项
  bestEffort?: boolean;          // 最佳努力模式（失败不重试）
  gifPlayback?: boolean;         // GIF 播放模式
  silent?: boolean;              // 静默投递（不触发通知）

  // 镜像通知
  mirror?: DeliveryMirrorPayload;
};

type QueuedDelivery = QueuedDeliveryPayload & {
  id: string;                    // 投递唯一标识
  enqueuedAt: number;            // 入队时间戳
  retryCount: number;            // 已重试次数
  lastAttemptAt?: number;        // 上次尝试时间
  lastError?: string;            // 上次失败原因
};

type DeliveryMirrorPayload = {
  sessionKey: string;
  agentId: string;
  text: string;
  mediaUrls: string[];
};
```

Listing 41: 队列消息完整定义

13.3 原子性模型

投递队列使用三阶段原子操作确保数据一致性：

```typescript
// 阶段 1: 入队（写入临时文件后重命名）
async function enqueueDelivery(payload: QueuedDeliveryPayload) {
  const id = generateUniqueId();
  const tempPath = `${stateDir}/${id}.tmp`;
  const jsonPath = `${stateDir}/${id}.json`;

  // 先写临时文件
  await fs.promises.writeFile(tempPath, JSON.stringify(delivery));
  // 原子重命名
  await fs.promises.rename(tempPath, jsonPath);
}

// 阶段 2: 确认投递
async function ackDelivery(id: string) {
  const jsonPath = `${stateDir}/${id}.json`;
  const deliveredPath = `${stateDir}/${id}.delivered`;

  // 原子重命名为 .delivered
```

```
  await fs.promises.rename(jsonPath, deliveredPath);
  // 删除标记文件
  await fs.promises.unlink(deliveredPath);
}

// 阶段 3: 崩溃恢复（启动时）
async function recoverPendingDeliveries() {
  // 扫描 .json 文件 → 重新入队
  // 清理残留 .delivered 文件
  // 清理残留 .tmp 文件
}
```

Listing 42: 三阶段原子操作

13.4 指数退避重试

Table 19: 重试时间表

重试次数	等待时间	累计时间
第 1 次	5 秒	5 秒
第 2 次	25 秒	30 秒
第 3 次	2 分钟	2 分 30 秒
第 4 次	10 分钟	12 分 30 秒
第 5 次	—	最终失败

```
const BACKOFF_SCHEDULE_MS = [
  5_000,       // 5 秒
  25_000,      // 25 秒
  120_000,     // 2 分钟
  600_000,     // 10 分钟
];

function getNextRetryDelay(retryCount: number): number | null {
  if (retryCount >= BACKOFF_SCHEDULE_MS.length) {
    return null; // 超过最大重试次数，放弃
  }
  return BACKOFF_SCHEDULE_MS[retryCount];
}
```

Listing 43: 退避时间表实现

13.5 bestEffort 模式

- 当 `bestEffort: true` 时，投递失败不会触发重试
- 适用于非关键消息（如心跳、状态通知）
- 减少队列压力，避免非关键消息阻塞重要消息

13.6 镜像通知

镜像机制允许将 Agent 的回复同时发送到其他渠道：

```
// 当 Agent 回复 Telegram 消息时，
// 可以同时将回复镜像到 Web UI
mirror: {
  sessionKey: "telegram:chat:12345",
  agentId: "default",
  text: "Agent 的回复文本",
  mediaUrls: ["https://..."],
}
```

Listing 44: 镜像投递

13.7 心跳过滤

对于流式输出的心跳消息，系统会根据渠道的可见性设置进行过滤：

- 某些渠道不支持流式更新（如 SMS）
- 心跳频率可按渠道配置
- 内联指令标签（directive tags）在发送前被剥离

14 Agent 智能体系统

14.1 子系统概览

Agent 系统是 OpenClaw 中最大的子系统，包含约 538 个源文件，负责 AI 智能体的编排、执行和管理。

Table 20: Agent 子系统模块

模块	文件数	职责
`agents/`	538	核心编排、工作区管理、Auth Profile
`context-engine/`	–	上下文构建与注入
`auto-reply/`	–	自动回复、模板、调度
`providers/`	13	LLM 模型提供商适配
`acp/`	31	Agent Control Protocol

14.2 Agent 作用域解析

`agents/agent-scope.ts` 负责将会话键映射到正确的 Agent：

```
function resolveSessionAgentIds(params: {
  sessionKey?: string;
  config?: OpenClawConfig;
  agentId?: string;
}): {
  defaultAgentId: string;     // 配置中的默认 Agent
  sessionAgentId: string;     // 实际用于此会话的 Agent
} {
  // 1. 规范化 Agent ID（转小写）
  // 2. 从 SessionKey 中提取嵌入的 Agent ID
  // 3. 如果未嵌入，使用配置中的默认值
  // 4. 追踪并记录默认 Agent 警告（仅一次）
}
```

Listing 45: Agent 作用域解析

14.3 多智能体工作区隔离

每个 Agent 拥有独立的工作区：

```
~/.openclaw/agents/
  |-- default/                # 默认 Agent
  |   |-- sessions/           # 会话存储
  |   |   |-- telegram_chat_123.jsonl
  |   |   |-- discord_guild_456.jsonl
  |   |-- memory/             # 向量记忆
  |   |-- config.json         # Agent 专属配置
  |   |-- skills/             # 绑定的技能
  |
  |-- work-agent/             # 工作用 Agent
  |   |-- sessions/
  |   |-- memory/
  |   |-- config.json
```

Listing 46: Agent 工作区目录结构

14.4 Auth Profile 管理

`agents/auth-profiles/` 实现了 LLM API 凭据的智能管理：

```
type AuthProfile = {
  id: string;
  provider: string;           // "anthropic", "openai", "google", ...
  model?: string;             // 指定模型
```

```
  apiKey: string;             // API 密钥

  // 状态追踪
  lastUsedAt?: number;
  failureCount: number;
  lastFailureAt?: number;
  cooldownUntil?: number;     // 冷却截止时间

  // 配额
  rateLimitResetAt?: number;
  dailyBudget?: number;
  dailyUsage?: number;
};

// 选择逻辑:
// 1. 排除处于冷却中的 Profile
// 2. 排除超出每日预算的 Profile
// 3. 按失败次数排序(优先使用健康的)
// 4. 轮换使用(Round-robin)
```

Listing 47: Auth Profile 机制

冷却与降级策略:

- 速率限制冷却:收到 429 后,等待 `Retry-After` 指定时间
- 错误冷却:连续失败后进入指数退避冷却
- 自动过期:长时间未使用的 Profile 自动失效
- 降级:主 Profile 不可用时,自动切换到备用 Profile

14.5 Pi 嵌入式运行器

`agents/pi-embedded-runner/runs.ts` 管理嵌入式 Agent 运行的生命周期:

```
// 全局注册表
const ACTIVE_EMBEDDED_RUNS = new Map<string, EmbeddedPiQueueHandle>();
const EMBEDDED_RUN_WAITERS = new Map<string, Set<EmbeddedRunWaiter>>();

// 公共 API
export function queueEmbeddedPiMessage(
  sessionId: string,
  text: string,
): boolean;

export function abortEmbeddedPiRun(
  sessionId: string,
): boolean;

export function waitForEmbeddedPiRunEnd(
  sessionId: string,
  timeoutMs = 15_000,
): Promise<boolean>;

export function setActiveEmbeddedRun(
  sessionId: string,
  handle: EmbeddedPiQueueHandle,
  sessionKey?: string,
);

export function clearActiveEmbeddedRun(
  sessionId: string,
  handle: EmbeddedPiQueueHandle,
  sessionKey?: string,
);
```

Listing 48: Pi 嵌入式运行器 API

14.6 上下文引擎

`context-engine/` 负责为每次 Agent 调用构建完整的上下文：

1. 系统提示：Agent 的基础指令和人设
2. 会话历史：从 JSONL 加载的对话历史
3. 记忆检索：通过 Memory 系统检索相关记忆
4. 工具描述：绑定到 Agent 的工具列表
5. 技能注入：活跃技能的上下文信息
6. **Hook** 注入：`beforePromptBuild` Hook 的额外上下文

14.7 自动回复系统

`auto-reply/` 实现了消息的自动分发和回复逻辑：

```typescript
// reply/dispatcher-registry.ts
type DispatcherRegistry = {
  // 追踪每个会话/线程的待处理回复
  pending: Map<string, PendingReply>;

  // 管理并发（防止同一会话的并发回复）
  locks: Map<string, Promise<void>>;

  // 注册和触发回复
  dispatch: (sessionKey, message) => Promise<void>;
  cancel: (sessionKey) => void;
};
```

Listing 49: 自动回复调度器

14.8 模型提供商

支持 13 种 LLM 模型提供商：

Table 21: 支持的 LLM 提供商

提供商	支持的模型系列
Anthropic	Claude (Opus, Sonnet, Haiku)
OpenAI	GPT-4, GPT-4o, o1, o3
Google	Gemini Pro, Gemini Ultra
AWS Bedrock	多模型（Anthropic, Meta, Mistral）
Azure OpenAI	GPT-4 系列
Mistral	Mistral Large, Medium
Ollama	本地模型
Groq	高速推理
Together AI	开源模型
DeepSeek	DeepSeek 系列
通义千问	Qwen 系列
Minimax	Minimax 系列
自定义	OpenAI 兼容接口

15 工具系统与执行引擎

15.1 工具类型定义

```
type AnyAgentTool = {
  name: string;
  description: string;
  input_schema?: {
    type: "object";
    properties: Record<string, unknown>;
    required?: string[];
  };
  execute: (input: unknown) => Promise<
    string | { type: string; content: unknown }
  >;

  // 扩展属性
  ownerOnly?: boolean;       // 仅所有者可调用
  requiresApproval?: boolean; // 需要审批
};
```

Listing 50: Agent 工具类型

15.2 工具分类

Table 22: 工具类型分类

类型	来源	示例
Agent 工具	Python SDK 集成	自定义脚本、API 调用
渠道工具	渠道适配器导出	发送消息、管理群组
系统工具	核心内置	浏览器、Canvas、文件操作
插件工具	插件注册	第三方服务集成
技能工具	技能绑定	GitHub、Notion 等

15.3 浏览器自动化

`src/browser/` 模块（135 源文件）基于 Playwright 实现浏览器自动化：

Table 23: 浏览器自动化核心文件

文件	职责
`client-actions.ts`	浏览器控制动作（导航、点击、输入等）
`server-context.tab-selection.ts`	标签页管理与选择
`server-lifecycle.ts`	浏览器进程生命周期
`extension-relay-auth.ts`	浏览器扩展认证中继

浏览器工具的能力：

- 网页导航和内容提取
- 表单填写和交互
- 截图和视觉理解
- JavaScript 执行
- Cookie 和会话管理
- 多标签页管理

15.4 HTTP 工具调用

gateway/tools-invoke-http.ts 实现了通过 HTTP 触发工具执行：

```
// 请求/响应流式处理
async function invokeToolViaHttp(params: {
  tool: AnyAgentTool;
  input: unknown;
  sessionKey: string;
  agentId: string;
  runId: string;
}): Promise<ToolInvocationResult> {
  // 1. 验证工具输入参数
  // 2. 检查执行审批（如果需要）
  // 3. 调用工具 execute 方法
  // 4. 流式返回结果
  // 5. 错误处理与超时控制
}
```

Listing 51: HTTP 工具调用

15.5 工具事件接收器

工具执行过程中的事件通过接收器注册表分发：

```
type ToolEventRecipientRegistry = {
  // 注册事件接收器
  register: (
    runId: string,
    recipient: ToolEventRecipient,
  ) => void;

  // 注销事件接收器
  unregister: (runId: string) => void;

  // 发送事件到所有接收器
  emit: (
    runId: string,
    event: ToolEvent,
  ) => void;
};

// 工具事件类型
type ToolEvent =
  | { type: "start"; tool: string; input: unknown }
  | { type: "progress"; data: unknown }
  | { type: "result"; output: unknown }
  | { type: "error"; error: string };
```

Listing 52: 工具事件注册

15.6 执行审批

敏感工具的执行需要通过审批流程：

```
// 1. 工具标记为 requiresApproval
// 2. 执行前发送审批请求到网关
// 3. Web UI / 移动端展示审批界面
// 4. 用户批准或拒绝
// 5. 批准后继续执行，拒绝则返回错误

type ExecApproval = {
  id: string;
  tool: string;
  input: unknown;
  sessionKey: string;
  requestedAt: number;
  status: "pending" | "approved" | "denied";
  decidedAt?: number;
  decidedBy?: string;
};
```

Listing 53: 执行审批流程

15.7 Canvas Host

`canvas-host/` 提供了可视化工作空间的宿主环境：

- 独立 HTTP 端口运行
- 支持 HTML/CSS/JS 渲染
- 与 macOS/iOS 原生 A2UI 集成
- 能力 Token 认证
- 双向通信通道

15.8 Node Host

`node-host/`（18 源文件）提供 Node.js 代码执行环境：

- 沙箱化执行
- 模块导入控制
- 超时和资源限制
- 标准输入/输出捕获

16 插件系统架构

16.1 插件定义

```
type OpenClawPluginDefinition = {
  id?: string;           // 插件唯一标识
  name?: string;         // 显示名称
  version?: string;      // 版本号

  // 注册项
  tools?: OpenClawPluginToolFactory[];
  hooks?: PluginHookRegistration[];
  commands?: PluginCommandDefinition[];
  services?: PluginServiceDefinition[];
  channels?: PluginChannelRegistration[];
  providers?: PluginProviderRegistration[];
  httpRoutes?: PluginHttpRouteRegistration[];
  cliCommands?: PluginCliRegistration[];
};

type PluginKind = "memory" | "context-engine";
```

Listing 54: 插件定义类型

16.2 八种注册类型

Table 24: 插件注册类型详解

类型	接口	用途
工具	`PluginToolRegistration`	向 Agent 注册可调用的工具
CLI 命令	`PluginCliRegistration`	添加 `openclaw <cmd>` 命令
HTTP 路由	`PluginHttpRouteRegistration`	注册 Webhook 和 HTTP 端点
渠道	`PluginChannelRegistration`	集成新的消息渠道
提供商	`PluginProviderRegistration`	注册认证提供商
Hook	`PluginHookRegistration`	监听生命周期事件
服务	`PluginServiceRegistration`	运行长驻后台服务
命令	`PluginCommandRegistration`	渠道内斜杠命令

16.3 插件注册表

`plugins/registry.ts` 管理所有已注册插件的元数据：

```
class PluginRegistry {
  private tools: PluginToolRegistration[] = [];
  private hooks: PluginHookRegistration[] = [];
  private channels: PluginChannelRegistration[] = [];
  private services: PluginServiceRegistration[] = [];
  // ... 其他注册类型

  register(plugin: OpenClawPluginDefinition) {
    // 1. 验证插件定义完整性
    // 2. 检查 ID 唯一性
    // 3. 按类型分类注册
    // 4. 通知监听器
  }

  unregister(pluginId: string) {
    // 1. 停止关联的服务
    // 2. 移除所有注册项
    // 3. 清理资源
  }

  getToolsForAgent(agentId: string): AnyAgentTool[] {
    // 根据 Agent 配置过滤可用工具
```

```
  }
}
```

Listing 55: 插件注册表管理

16.4 动态加载

`plugins/loader.ts` 实现插件的动态发现和加载：

```
async function loadPlugin(pluginPath: string): Promise<void> {
  // 1. 解析插件路径（本地/npm/git）
  // 2. 读取 package.json 和入口文件
  // 3. 使用 jiti 进行动态导入（ESM 兼容）
  // 4. 调用插件的 register() 函数
  // 5. 注入 PluginRuntime 上下文
  // 6. 记录加载结果和错误
}
```

Listing 56: 插件动态加载

16.5 插件运行时

`plugins/runtime/index.ts` 为每个插件创建隔离的运行时环境：

```
type PluginRuntime = {
  // 版本信息
  version: string;

  // 配置访问
  config: {
    get: () => OpenClawConfig;
    getAgentConfig: (agentId: string) => AgentConfig;
  };

  // 子 Agent 能力
  subagent: {
    spawn: (opts: SubagentOptions) => Promise<SubagentResult>;
    // 仅在网关请求作用域内可用
  };

  // 系统操作
  system: {
    exec: (cmd: string, opts?) => Promise<ExecResult>;
    readFile: (path: string) => Promise<string>;
  };

  // 媒体处理
  media: {
    processImage: (path: string) => Promise<Buffer>;
    extractText: (path: string) => Promise<string>;
  };

  // 语音
  tts: { textToSpeechTelephony: Function };
  stt: { transcribeAudioFile: Function };

  // 工具
  tools: {
    invoke: (name: string, input: unknown) => Promise<unknown>;
  };

  // 渠道
  channel: {
    send: (target: OutboundTarget, payload: ReplyPayload) => Promise<void>;
  };

  // 事件
  events: {
    emit: (event: string, payload: unknown) => void;
    on: (event: string, handler: Function) => void;
  };
```

```
  // 日志
  logging: {
    debug: (msg: string) => void;
    info: (msg: string) => void;
    warn: (msg: string) => void;
    error: (msg: string) => void;
  };

  // 状态
  state: {
    resolveStateDir: (pluginId: string) => string;
  };
};
```

Listing 57: PluginRuntime 完整接口

16.6 Plugin SDK

Plugin SDK 提供 40+ 个独立导出入口，支持 Tree-shaking：

```
// 核心
openclaw/plugin-sdk            // 主入口
openclaw/plugin-sdk/core       // 核心类型和工具

// 渠道特定
openclaw/plugin-sdk/telegram   // Telegram 渠道 API
openclaw/plugin-sdk/discord    // Discord 渠道 API
openclaw/plugin-sdk/slack      // Slack 渠道 API
openclaw/plugin-sdk/signal     // Signal 渠道 API
openclaw/plugin-sdk/line       // LINE 渠道 API

// 功能模块
openclaw/plugin-sdk/memory     // 记忆系统 API
openclaw/plugin-sdk/tools      // 工具定义 API
openclaw/plugin-sdk/acp        // Agent Control Protocol
openclaw/plugin-sdk/test-utils // 测试工具
```

Listing 58: SDK 导出结构

16.7 插件包结构规范

每个扩展插件遵循统一的包结构：

```
extensions/<name>/
  |-- package.json       # 独立依赖声明
  |-- tsconfig.json      # TypeScript 配置
  |-- index.ts           # 插件入口（注册逻辑）
  |-- src/
  |    |-- plugin.ts     # 插件定义
  |    |-- channel.ts    # 渠道实现（如果是渠道插件）
  |    |-- commands.ts   # 命令注册
  |    |-- hooks.ts      # Hook 注册
  |    |-- ...
  |-- test/
  |    |-- *.test.ts     # 测试文件
```

Listing 59: 标准插件包结构

17 Hook 事件驱动系统

17.1 Hook 执行框架

plugins/hooks.ts 实现了两种 Hook 执行模式：

```
function createHookRunner(
  registry: PluginRegistry,
  options: HookRunnerOptions = {},
) {
  // Void Hook: 并行执行，无返回值
  async function runVoidHook<K extends PluginHookName>(
    hookName: K,
    event: HookEventType<K>,
    ctx: HookContextType<K>,
  ): Promise<void> {
    const hooks = getHooksForName(registry, hookName);

    // 所有 Hook 并行执行
    const promises = hooks.map(async (hook) => {
      try {
        await hook.handler(event, ctx);
      } catch (err) {
        handleHookError({
          hookName,
          pluginId: hook.pluginId,
          error: err,
        });
      }
    });

    await Promise.all(promises);
  }

  // Modifying Hook: 串行执行，结果累积
  async function runModifyingHook<K extends PluginHookName, TResult>(
    hookName: K,
    event: HookEventType<K>,
    ctx: HookContextType<K>,
    mergeResults?: (accumulated?: TResult, next: TResult) => TResult,
  ): Promise<TResult | undefined> {
    const hooks = getHooksForName(registry, hookName);

    // 按优先级排序后串行执行
    hooks.sort((a, b) => (b.priority ?? 0) - (a.priority ?? 0));

    let result: TResult | undefined;
    for (const hook of hooks) {
      try {
        const next = await hook.handler(event, ctx);
        if (mergeResults && next !== undefined) {
          result = mergeResults(result, next as TResult);
        }
      } catch (err) {
        handleHookError({
          hookName,
          pluginId: hook.pluginId,
          error: err,
        });
      }
    }
    return result;
  }

  return { runVoidHook, runModifyingHook };
}
```

Listing 60: Hook 运行器创建

17.2 完整生命周期事件

Table 25: Hook 生命周期事件完整列表

事件名	类型	触发时机与用途
gatewayStart	Void	网关启动完成后触发，用于初始化插件资源
gatewayStop	Void	网关关闭前触发，用于清理插件资源
sessionStart	Void	新会话创建时触发
sessionEnd	Void	会话结束时触发
beforeAgentStart	Modifying	Agent 开始执行前，可修改 Agent 配置
agentEnd	Void	Agent 执行结束后触发
beforeModelResolve	Modifying	模型解析前，可动态切换模型
beforePromptBuild	Modifying	提示词构建前，可注入额外上下文
beforeToolCall	Modifying	工具调用前，可拦截或修改参数
afterToolCall	Void	工具调用后，可记录或分析结果
beforeMessageWrite	Modifying	消息写入会话前，可修改内容
messageSending	Modifying	消息发送前，可修改或拦截
messageSent	Void	消息成功发送后触发
subagentSpawning	Modifying	子 Agent 创建前，可修改配置
subagentSpawned	Void	子 Agent 创建成功后触发
subagentDeliveryTarget	Modifying	子 Agent 投递目标解析
subagentEnded	Void	子 Agent 结束时触发
beforeCompaction	Modifying	会话压缩前，可调整压缩策略
afterCompaction	Void	会话压缩完成后触发
beforeReset	Void	会话重置前触发

17.3 Hook 优先级

- Modifying Hook 按优先级降序执行（高优先级先执行）
- 默认优先级为 0
- 同优先级的 Hook 按注册顺序执行
- Void Hook 不受优先级影响（并行执行）

17.4 Hook 来源分类

Table 26: Hook 来源

来源	说明
openclaw-bundled	内置 Hook，随核心分发
openclaw-managed	系统管理的 Hook
openclaw-workspace	用户工作区自定义 Hook
openclaw-plugin	第三方插件注册的 Hook

17.5 内置 Hook 详解

17.5.1 boot-md Hook

启动时加载 BOOT.md 配置文件，为 Agent 注入启动指令：

```
// hooks/bundled/boot-md/handler.ts
// 事件: beforePromptBuild
// 功能: 读取 Agent 工作区中的 BOOT.md
//       将内容注入到系统提示中
```

Listing 61: boot-md Hook

17.5.2 session-memory Hook

```
// hooks/bundled/session-memory/handler.ts
// 事件: sessionStart, agentEnd
// 功能:
//   sessionStart: 预热记忆索引
//   agentEnd: 将重要对话保存到记忆
```

Listing 62: session-memory Hook

17.5.3 command-logger Hook

```
// hooks/bundled/command-logger/handler.ts
// 事件: afterToolCall
// 功能: 记录所有工具调用到日志
//       用于审计和调试
```

Listing 63: command-logger Hook

17.6 错误处理

Hook 执行中的错误不会中断主流程:

```
function handleHookError(params: {
  hookName: string;
  pluginId: string;
  error: unknown;
}) {
  // 1. 记录错误日志 (不抛出)
  // 2. 发送诊断事件
  // 3. 统计失败次数
  // 4. 超过阈值时禁用 Hook
}
```

Listing 64: Hook 错误处理

18 Memory 记忆与向量搜索

18.1 类继承层次

Memory 系统采用模板方法模式，通过三层继承实现代码复用：

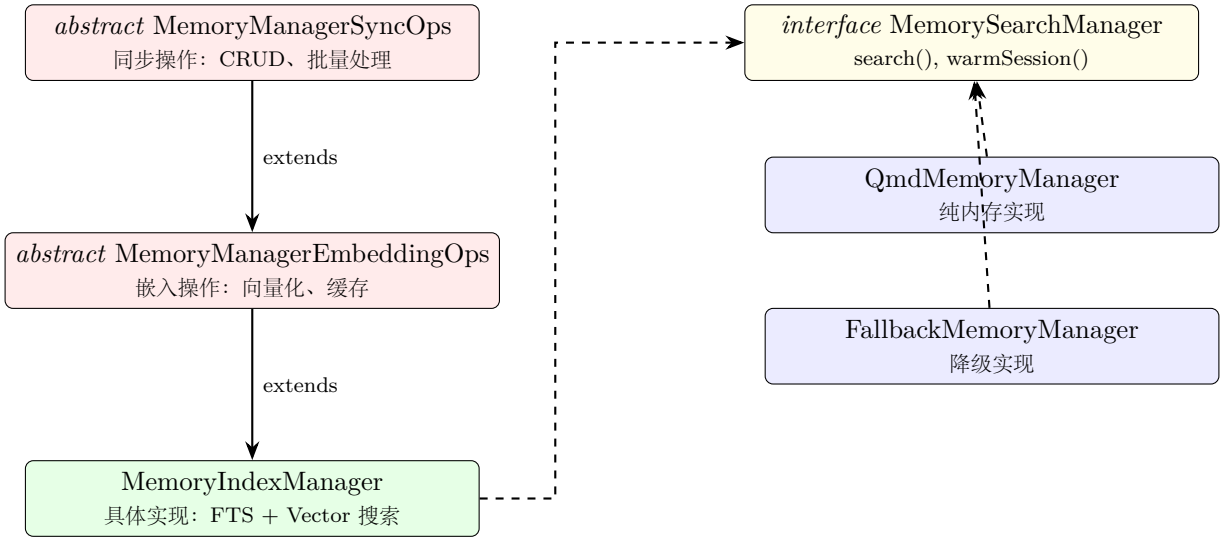

Figure 6: Memory 系统类继承层次

18.2 MemorySearchManager 接口

```
interface MemorySearchManager {
  search(
    query: string,
    opts?: {
      maxResults?: number;     // 最大结果数
      minScore?: number;       // 最低相关度分数
      sessionKey?: string;     // 限定会话范围
    },
  ): Promise<MemorySearchResult[]>;

  warmSession(sessionKey: string): Promise<void>;
}

type MemorySearchResult = {
  id: string;
  text: string;
  score: number;             // 相关度分数 [0, 1]
  source: string;            // 来源标识
  createdAt: number;         // 创建时间
  metadata?: Record<string, unknown>;
};
```

Listing 65: 记忆搜索接口

18.3 MemoryIndexManager 实现

```
class MemoryIndexManager extends MemoryManagerEmbeddingOps
  implements MemorySearchManager {

  // 单例缓存：按 agentId + workspaceDir + settings 键
  private static cache = new Map<string, MemoryIndexManager>();
  private static pending = new Map<string, Promise<MemoryIndexManager>>();

  static async get(params: {
    cfg: OpenClawConfig;
    agentId: string;
    purpose?: "default" | "status";
```

```
  }): Promise<MemoryIndexManager | null> {
    const key = `${params.agentId}:${workspaceDir}:${settingsHash}`;

    // 检查缓存
    if (this.cache.has(key)) return this.cache.get(key)!;

    // 检查是否有待处理的初始化
    if (this.pending.has(key)) return this.pending.get(key)!;

    // 创建新实例
    const promise = this.create(params);
    this.pending.set(key, promise);
    const instance = await promise;
    this.cache.set(key, instance);
    this.pending.delete(key);
    return instance;
  }
}
```

Listing 66: 单例模式的索引管理器

18.4 嵌入向量提供商

Table 27: 支持的嵌入模型提供商

提供商	模型	特点
OpenAI	text-embedding-3-*	支持批量处理、高质量
Google Gemini	embedding-001	Google 生态集成
Voyage AI	voyage-*	专注检索优化
Mistral AI	mistral-embed	开源友好
Ollama	本地模型	完全离线、隐私优先
自定义	—	OpenAI 兼容接口

18.5 存储层

Memory 使用 SQLite + sqlite-vec 扩展作为存储后端：

```
-- 向量嵌入表
CREATE TABLE chunks_vec (
  id TEXT PRIMARY KEY,
  embedding BLOB,          -- 向量数据（float32 数组）
  dimension INTEGER        -- 向量维度
);

-- 全文检索表
CREATE TABLE chunks_fts (
  id TEXT PRIMARY KEY,
  text TEXT,               -- 原文内容
  source TEXT,             -- 来源标识
  created_at INTEGER,      -- 创建时间
  session_key TEXT,        -- 关联会话
  metadata TEXT            -- JSON 元数据
);

-- 嵌入缓存表（避免重复计算）
CREATE TABLE embedding_cache (
  text_hash TEXT PRIMARY KEY,
  provider TEXT,
  model TEXT,
  embedding BLOB,
  created_at INTEGER
);
```

Listing 67: 数据库表结构

18.6 混合搜索

memory/hybrid.ts 实现了 BM25 + 向量相似度的混合搜索：

```
async function mergeHybridResults(params: {
  vector: HybridVectorResult[];      // 向量搜索结果
  keyword: HybridKeywordResult[];    // BM25 关键词结果
  vectorWeight: number;              // 向量权重（默认 0.7）
  textWeight: number;                // 文本权重（默认 0.3）
  mmr?: Partial<MMRConfig>;          // 多样性重排序
  temporalDecay?: Partial<TemporalDecayConfig>;  // 时间衰减
}): Promise<MergedResult[]> {

  // 1. BM25 分数归一化
  // relevance / (1 + relevance) -> [0, 1]
  const normalizedKeyword = keyword.map(r => ({
    ...r,
    score: r.relevance / (1 + r.relevance),
  }));

  // 2. 加权合并
  // finalScore = vectorWeight * v_score + textWeight * t_score
  const merged = combineResults(vector, normalizedKeyword);

  // 3. MMR 多样性重排序（可选）
  if (mmr) {
    applyMMR(merged, {
      lambda: mmr.lambda ?? 0.5,      // 相关性-多样性平衡
      maxResults: mmr.maxResults ?? 10,
    });
  }

  // 4. 时间衰减（可选）
  if (temporalDecay) {
    applyTemporalDecay(merged, {
      halfLife: temporalDecay.halfLife ?? 7 * 24 * 3600 * 1000, // 7天
      reference: Date.now(),
    });
  }

  return merged.sort((a, b) => b.score - a.score);
}
```

Listing 68: 混合搜索合并策略

18.7 同步触发器

记忆索引在以下时机触发同步：

Table 28: 记忆同步触发器

触发器	说明
会话启动	warmSession() 预热索引
搜索时	如果索引标记为 dirty，先同步再搜索
定时	可配置的定时同步间隔
文件监听	监控记忆源文件变更
Agent 结束	session-memory Hook 保存重要对话

19 配置管理与热重载

19.1 配置加载

src/config/config.ts 提供完整的配置生命周期管理：

```
// 加载并验证配置
export function loadConfig(): OpenClawConfig;

// 读取配置快照（含验证结果和遗留问题）
export function readConfigFileSnapshot(): ConfigFileSnapshot;

// 原子写入配置
export function writeConfigFile(config: OpenClawConfig): void;

// 运行时验证
export function validateConfigObject(obj: unknown): ValidationResult;

// 旧版配置自动迁移
export function migrateLegacyConfig(obj: unknown): OpenClawConfig;

// JSON5 解析（支持注释）
export function parseConfigJson5(text: string): unknown;
```

Listing 69: 配置 API

19.2 ConfigFileSnapshot 结构

```
type ConfigFileSnapshot = {
  exists: boolean;              // 配置文件是否存在
  valid: boolean;               // 验证是否通过
  parsed: OpenClawConfig;       // 解析后的配置对象
  issues: ConfigIssue[];        // 验证错误/警告
  legacyIssues: ConfigIssue[]; // 需要迁移的遗留问题
  hash?: string;                // 文件内容哈希（变更检测）
};

type ConfigIssue = {
  severity: "error" | "warning" | "info";
  path: string;                 // 配置路径（如 "telegram.token"）
  message: string;              // 问题描述
  suggestion?: string;          // 修复建议
};
```

Listing 70: 配置快照

19.3 JSON5 配置格式

OpenClaw 使用 JSON5 作为配置文件格式，相比标准 JSON 的优势：

```
{
  // 网关配置
  gateway: {
    port: 18789,
    auth: "token",          // 认证模式
    host: "127.0.0.1",      // 绑定地址
  },

  // Telegram 配置
  telegram: {
    accounts: {
      main: {
        token: "BOT_TOKEN_HERE",
        mode: "polling",  // 或 "webhook"
        allowlist: [123456789],
      },
    },
  },

  // Agent 默认配置
```

```
  agents: {
    default: {
      model: "claude-sonnet-4-6",
      skills: ["github", "notion"],
    },
  },
}
```

Listing 71: config.json 示例

19.4 配置层次

配置按以下优先级层叠覆盖（后者覆盖前者）：

1. `.env` / `.env.local` — 环境变量
2. `~/.openclaw/config.json` — 用户配置文件
3. CLI 参数和 API 运行时覆盖
4. 渠道专属认证 Token（独立存储）

19.5 Zod Schema 验证

所有配置均通过 Zod Schema 进行类型安全验证：

```
// config/types.telegram.ts
const TelegramAccountSchema = z.object({
  token: z.string(),
  mode: z.enum(["polling", "webhook"]).default("polling"),
  webhookUrl: z.string().url().optional(),
  allowlist: z.array(z.number()).default([]),
  groupAllowlist: z.array(GroupAllowlistEntrySchema).default([]),
  dmPolicy: z.enum(["pairing", "open"]).default("pairing"),
});

// config/types.gateway.ts
const GatewayConfigSchema = z.object({
  port: z.number().int().min(1).max(65535).default(18789),
  host: z.string().default("127.0.0.1"),
  auth: z.enum(["token", "none"]).default("token"),
});
```

Listing 72: 配置 Schema 示例

19.6 热重载机制

19.6.1 文件监听

```
// config-reload.ts
import { watch } from "chokidar";

const watcher = watch(configFilePath, {
  persistent: true,
  ignoreInitial: true,
  awaitWriteFinish: {
    stabilityThreshold: 300,  // 防抖 300ms
    pollInterval: 100,
  },
});

watcher.on("change", () => {
  scheduleReload();
});
```

Listing 73: 配置文件监听

19.6.2 重载计划构建

```
function buildGatewayReloadPlan(
  prevConfig: OpenClawConfig,
  nextConfig: OpenClawConfig,
): ReloadPlan {
  // 1. 计算配置差异路径
  const changedPaths = diffConfigPaths(prevConfig, nextConfig);

  // 2. 根据路径判断重载类型
  // secrets.* -> 需要重启
  // memory.* -> 热重载
  // gateway.auth.* -> 热重载
  // telegram.accounts.* -> 渠道重启
  // agents.* -> Agent 重载

  return {
    requiresRestart: hasRestartPaths(changedPaths),
    channelsToRestart: getChangedChannels(changedPaths),
    configToReload: getReloadableConfig(changedPaths),
  };
}
```

Listing 74: 重载计划

19.6.3 递归差异比较

```
function diffConfigPaths(
  prev: unknown,
  next: unknown,
  prefix = "",
): string[] {
  // 递归比较对象和数组
  // 返回变更路径列表:
  // ["gateway.auth", "telegram.accounts.main.token",
  //  "plugins.list[0]"]
}
```

Listing 75: 配置差异比较

19.6.4 重载状态机

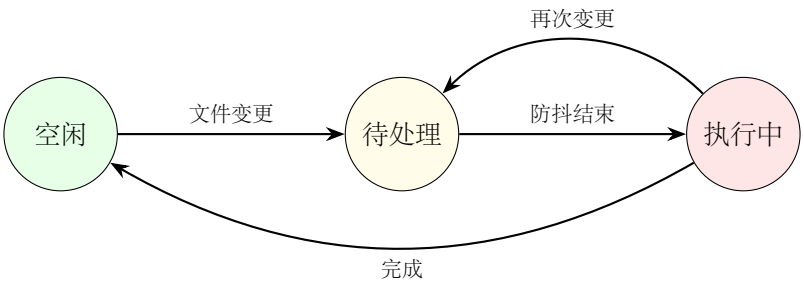

Figure 7: 配置重载状态机

20 安全机制与凭据管理

20.1 安全体系总览

OpenClaw 的安全体系覆盖多个层面：

Figure 8: 五层安全体系

20.2 凭据管理

20.2.1 存储结构

凭据存储在加密的本地目录中：

```
~/.openclaw/credentials/
  |-- auth-profiles/        # Auth Profile 存储
  |    |-- anthropic.json   # Anthropic API Key
  |    |-- openai.json      # OpenAI API Key
  |    |-- google.json      # Google API Key
  |
  |-- channel-tokens/       # 渠道 Token 存储
  |    |-- telegram.json
  |    |-- discord.json
  |
  |-- audit.log             # 审计日志
```

Listing 76: 凭据存储目录

20.2.2 secrets/apply.ts 凭据申请

```
async function applySecrets(params: {
  config: OpenClawConfig;
  secretsDir: string;
}): Promise<ApplyResult> {
  // 1. 读取当前文件快照
  const snapshot = await readCurrentSnapshot(params.secretsDir);

  // 2. 比较差异
  const diff = compareSnapshots(snapshot, params.config);

  // 3. 更新 Auth Profile 存储
  for (const change of diff.authProfileChanges) {
    await updateAuthProfile(change);
  }

  // 4. 更新环境变量
  for (const envChange of diff.envChanges) {
```

```
    await updateEnvVar(envChange);
  }

  // 5. 原子写入（带回滚）
  try {
    await atomicWrite(params.secretsDir, updatedSnapshot);
  } catch (error) {
    await rollback(snapshot);
    throw error;
  }

  return { applied: diff.totalChanges };
}
```

Listing 77: 凭据申请流程

20.3 DM 策略

每个渠道账户可配置两种消息接收策略：

Table 29: DM 策略模式

策略	行为
`pairing`（默认）	需要通过 QR 码或配对码完成设备配对后才能交互。最安全，适合个人使用。
`open`	开放接收消息，配合白名单使用。适合群组机器人或公共服务场景。

20.4 白名单验证

`security/audit-channel.ts` 实现渠道白名单的审计和验证：

```
function auditChannelAllowlist(params: {
  channel: ChannelId;
  allowlist: AllowlistEntry[];
}): AuditResult {
  const findings: AuditFinding[] = [];

  for (const entry of params.allowlist) {
    // Discord: 基于用户名的条目是可变的（不安全）
    if (channel === "discord" && isNameBasedEntry(entry)) {
      findings.push({
        severity: "warning",
        message: "Discord name-based entries are mutable",
        suggestion: "Use numeric user IDs instead",
      });
    }

    // Telegram: 验证数字 ID 格式
    if (channel === "telegram" && !isValidNumericId(entry)) {
      findings.push({
        severity: "error",
        message: "Invalid Telegram numeric ID",
      });
    }

    // 检测无效条目
    if (isInvalidEntry(entry)) {
      findings.push({
        severity: "error",
        message: `Invalid allowlist entry: ${entry}`,
      });
    }
  }

  return { findings, isSecure: findings.length === 0 };
}
```

Listing 78: 白名单审计

20.5 路径边界强制

`src/infra/boundary-path.ts`（~24,000 行）实现了严格的路径安全检查：

```
function enforcePathBoundary(
  requestedPath: string,
  allowedBoundaries: string[],
): boolean {
  // 1. 规范化路径（解析 .., symlink 等）
  const resolved = path.resolve(requestedPath);

  // 2. 检查是否在允许的边界内
  for (const boundary of allowedBoundaries) {
    if (resolved.startsWith(boundary)) {
      return true;
    }
  }

  // 3. 拒绝越界访问
  return false;
}

// 用途：防止插件和工具访问授权范围外的文件系统路径
// 例如：阻止插件读取 /etc/passwd
```

Listing 79: 路径边界检查

20.6 技能权限扫描

`security/skill-scanner.ts` 分析技能的权限需求：

```
function scanSkillPermissions(skillPath: string): SkillPermissions {
  // 1. 解析技能的 SKILL.md 声明
  // 2. 扫描代码中的危险操作
  //    - 文件系统访问
  //    - 网络请求
  //    - 子进程执行
  //    - 环境变量读取
  // 3. 生成权限报告
  return {
    filesystem: ["read", "write"],
    network: ["http", "https"],
    exec: ["gh"],  // 例如 GitHub CLI
    env: ["GITHUB_TOKEN"],
  };
}
```

Listing 80: 技能权限扫描

20.7 自动修复

`security/fix.ts` 提供安全问题的自动修复能力：

- 自动将可变的用户名白名单条目替换为数字 ID
- 检测并警告使用弱密码或默认令牌
- 自动轮换过期的 API 密钥
- 修复文件权限（如凭据文件权限过于宽松）

21 前端 UI 系统 (Lit Web Components)

21.1 技术选型

控制面板 Web UI 基于 **Lit Web Components** 构建，选择 Lit 而非 React/Vue 的原因是其轻量级和与 Web 标准的天然契合。

Table 30: UI 技术栈

技术	版本	用途
Lit	latest	Web Components 框架，声明式模板
@lit/context	latest	组件间上下文传递（替代 prop drilling）
@lit-labs/signals	latest	响应式信号状态管理
signal-polyfill	latest	TC39 Signals 提案 Polyfill
Vite	latest	开发服务器、HMR、静态资源构建
DOMPurify	latest	HTML 内容净化（防 XSS 攻击）
marked	latest	Markdown → HTML 渲染

21.2 主应用组件

`ui/src/ui/app.ts`（630 行）是整个 Web UI 的根组件：

```
// <openclaw-app> Web Component
@customElement("openclaw-app")
class OpenClawApp extends LitElement {

  // WebSocket 网关客户端
  private client: GatewayBrowserClient;

  // ===== 状态管理（@state 装饰器）=====

  // 聊天状态
  @state() messages: ChatMessage[] = [];
  @state() streamingText: string = "";
  @state() chatQueue: QueuedMessage[] = [];
  @state() attachments: Attachment[] = [];

  // 配置状态
  @state() configRaw: string = "";
  @state() configForm: FormConfig = {};
  @state() configValid: boolean = true;
  @state() configSchema: JSONSchema = {};

  // 渠道状态
  @state() channelSnapshots: ChannelAccountSnapshot[] = [];
  @state() whatsappQR: string | null = null;

  // Agent 状态
  @state() agents: AgentInfo[] = [];
  @state() toolsCatalog: ToolInfo[] = [];
  @state() skills: SkillInfo[] = [];

  // 会话与使用
  @state() sessions: SessionInfo[] = [];
  @state() usageTimeSeries: UsageDataPoint[] = [];

  // UI 状态
  @state() activeTab: string = "overview";
  @state() theme: "light" | "dark" = "light";
  @state() sidebarOpen: boolean = true;
  @state() splitRatio: number = 0.5;

  // 生命周期
  connectedCallback() {
    super.connectedCallback();
    this.connectToGateway();
    this.loadHistory();
```

```
    this.watchConfigChanges();
  }

  // 渲染委托
  render() {
    return renderApp(this);
  }
}
```

Listing 81: 主应用组件

21.3 控制器层

UI 的业务逻辑被组织为 12 个控制器模块：

Table 31: UI 控制器模块

控制器	职责
`chat.ts`	加载消息历史、发送消息（含静默回复检测）、中止运行、管理流式输出
`channels.ts`	加载渠道状态快照、WhatsApp QR 登录/等待/登出、Nostr 配置编辑
`agents.ts`	加载 Agent 列表、工具目录、技能列表、保存 Agent 配置
`config.ts`	配置验证、表单/原始 JSON 模式切换、Schema 加载
`cron.ts`	定时任务创建/编辑/删除、运行日志查看
`sessions.ts`	会话列表查看、按渠道/Agent/时间过滤
`usage.ts`	Token 使用量/成本分析、时间序列图表
`skills.ts`	技能状态查看、技能编辑、技能消息测试
`devices.ts`	设备配对管理、QR 码显示
`logs.ts`	结构化日志查看、按级别/模块过滤、日志导出
`presence.ts`	用户在线状态追踪
`exec-approvals.ts`	工具执行审批流程管理

21.4 功能标签页

Table 32: UI 功能标签页

标签页	功能
Overview	系统总览：渠道连接状态、Agent 列表、近期活动
Chat	实时聊天界面：消息发送、流式输出显示、附件上传
Config	配置管理：表单编辑器 + 原始 JSON 编辑器、实时验证
Logs	结构化日志：按级别着色、按模块过滤、实时流
Cron	定时任务：Cron 表达式编辑、运行历史、下次触发时间
Approvals	安全审批：待处理的工具执行审批请求
Devices	设备管理：已配对设备列表、新设备配对

21.5 WebSocket 实时通信

UI 通过 `GatewayBrowserClient` 与网关保持实时连接：

```
class GatewayBrowserClient {
  private ws: WebSocket;

  // RPC 调用
  async request(method: string, params: unknown): Promise<unknown>;
```

```

  // 事件监听
  on(event: string, handler: (payload: unknown) => void): void;

  // 连接管理
  connect(url: string, token: string): Promise<void>;
  disconnect(): void;

  // 自动重连
  private reconnect(): void;
}
```

Listing 82: 浏览器 WebSocket 客户端

22 原生应用 (macOS/iOS/Android)

22.1 macOS 应用

22.1.1 技术栈

macOS 应用使用 **Swift + SwiftUI** 构建，采用 `@Observable` 宏（Observation 框架）进行状态管理。

22.1.2 AppState 核心状态

```
@Observable
class AppState {
  static let shared = AppState()

  // 网关连接
  var connectionMode: ConnectionMode  // unconfigured/local/remote
  var transportType: TransportType    // ssh/direct
  var gatewayURL: URL?

  // 用户偏好
  var isPaused: Bool
  var launchAtLogin: Bool
  var hasCompletedOnboarding: Bool
  var showDebugPane: Bool

  // 语音唤醒 (Swabble)
  var voiceWakeEnabled: Bool
  var voiceWakeTriggerWords: [String]

  // Canvas/Web 集成
  var canvasEnabled: Bool
  var webViewURL: URL?

  // 进程管理
  var gatewayProcess: Process?
  var isGatewayRunning: Bool
}
```

Listing 83: macOS AppState 单例

22.1.3 核心组件

Table 33: macOS 应用核心组件

文件	职责
`AppState.swift`	@Observable 单例，全局状态管理
`LaunchdManager.swift`	launchd 服务管理（安装/启动/停止）
`ConnectionModeResolver.swift`	网关连接模式自动检测
`ExecApprovalEvaluation.swift`	工具执行审批 UI
`WebChatSwiftUI.swift`	Web UI 嵌入（WKWebView）
`Constants.swift`	配置常量

22.1.4 功能特点

- 菜单栏应用：常驻菜单栏，快速访问
- **launchd** 集成：作为系统服务自动启动
- 语音唤醒：通过 Swabble 库实现语音触发
- **Canvas** 集成：原生 A2UI 可视化工作空间

- **SSH** 隧道：支持远程网关连接

22.2 iOS 应用

Table 34: iOS 应用特性

特性	说明
框架	SwiftUI
Canvas	A2UI 集成
相机	拍照和视觉理解
位置	精确/粗略位置获取
共享框架	OpenClawKit（与 macOS 共享）

22.3 Android 应用

22.3.1 架构

Android 应用使用 **Kotlin + Jetpack Compose** 构建，核心是 `NodeRuntime` 类：

```kotlin
class NodeRuntime {
  // Canvas（Web 视图控制器）
  val canvas: CanvasController

  // 硬件访问
  val camera: CameraCapture
  val microphone: MicrophoneInput
  val sms: SmsManager

  // 网关交互
  val gatewayDiscovery: GatewayDiscovery
  val gatewayConnection: GatewayConnection

  // 聊天
  val chatSessions: ChatSessionManager
  val messageStreaming: MessageStreamHandler

  // 语音
  val voiceInput: VoiceInputHandler
  val liveTranscription: TranscriptionService

  // 位置
  val locationTracker: LocationTracker
}
```

Listing 84: Android NodeRuntime 核心职责

22.3.2 核心界面

Table 35: Android 应用核心界面

组件	功能
`OnboardingFlow.kt`	首次使用引导流程
`CanvasScreen.kt`	主聊天/Canvas 界面
`CameraHudOverlay.kt`	相机拍照界面（含闪光灯）
`ChatMarkdown.kt`	Markdown 消息渲染
`GatewayConfigResolver.kt`	网关连接配置

22.3.3 独特功能

- 前后台状态追踪：感知应用前后台切换

- 实时转录：麦克风输入实时语音转文字
- **SMS** 拦截：读取和管理短信
- 相机闪光灯：拍照时的闪光灯控制
- 语音对话历史：记录语音交互日志
- 麦克风冷却：防止连续触发的冷却机制

22.4 共享框架：OpenClawKit

`apps/shared/OpenClawKit/` 提供了 Swift 共享代码：

- 网关协议类型定义
- WebSocket 客户端封装
- 设备身份管理
- 加密和签名工具
- 通用 UI 组件

23 扩展生态与技能系统

23.1 官方扩展分类

41 个官方扩展按功能分为以下类别：

23.1.1 消息渠道扩展（20 个）

Table 36: 消息渠道扩展

扩展	目录	特点
Discord	`discord/`	支持 Guild/DM、线程、反应、子 Agent Hook
Telegram	`telegram/`	多账户、Polling/Webhook、投票、Markdown 分块
Slack	`slack/`	Bolt 框架、Block Kit、线程、文件共享
Signal	`signal/`	E164 号码、群组、媒体字节限制
Matrix	`matrix/`	E2EE 加密运行时、异步密钥引导
iMessage	`imessage/`	AppleScript 桥接、BlueBubbles 备选
IRC	`irc/`	经典 IRC 协议、多频道
MS Teams	`msteams/`	Bot Framework、Activity 处理
Google Chat	`googlechat/`	Workspace API、Space 管理
飞书	`feishu/`	Lark SDK、消息卡片
LINE	`line/`	Messaging API、Rich Menu
Mattermost	`mattermost/`	自托管兼容、WebSocket 事件
Twitch	`twitch/`	IRC 聊天、命令响应
Nostr	`nostr/`	去中心化协议、Relay 连接
Nextcloud Talk	`nextcloud-talk/`	Nextcloud API、房间管理
Synology Chat	`synology-chat/`	Synology NAS 集成
Tlon	`tlon/`	Urbit 网络集成
Zalo	`zalo/`	Zalo OA API（越南）
Zalo User	`zalo-user/`	Zalo 个人账户
BlueBubbles	`bluebubbles/`	BlueBubbles API（iMessage 桥接）

23.1.2 功能扩展（21 个）

Table 37: 功能扩展

扩展	功能描述
`memory-core/`	核心记忆管理插件
`memory-lancedb/`	LanceDB 向量数据库后端（高性能）
`voice-call/`	语音通话支持
`talk-voice/`	语音交互引擎
`device-pair/`	设备配对协议实现
`diagnostics-otel/`	OpenTelemetry 可观测性集成
`llm-task/`	LLM 任务编排与批处理
`copilot-proxy/`	GitHub Copilot 代理
`open-prose/`	文本处理与优化
`phone-control/`	手机远程控制
`thread-ownership/`	线程所有权管理
`diffs/`	差异对比工具

lobster/	特殊功能集成
acpx/	ACP 扩展功能
google-gemini-cli-auth/	Google Gemini CLI 认证
minimax-portal-auth/	Minimax 门户认证
qwen-portal-auth/	通义千问门户认证
test-utils/	扩展测试工具包

23.2 技能系统

23.2.1 技能定义格式

每个技能通过 SKILL.md 文件进行声明：

```
---
name: github
description: GitHub repository operations
requires:
  - gh (GitHub CLI, authenticated)
capabilities:
  - issue management
  - pull request operations
  - CI status checks
  - code review
limitations:
  - Not for local git operations
  - Not for bulk operations
  - Not for complex diffs
---

# GitHub Skill

Uses the `gh` CLI for GitHub operations...
```

Listing 85: SKILL.md 格式示例（GitHub 技能）

23.2.2 技能分类

Table 38: 内置技能分类（53 个）

类别	技能	功能
开发工具	github coding-agent tmux	PR/Issue/CI 管理 代码分析与生成 终端多窗口控制
笔记与文档	notion apple-notes bear-notes	Notion 页面/数据库操作 Apple Notes 读写 Bear 笔记应用集成
通信	himalaya imsg	邮件收发 iMessage 消息管理
生产力	apple-reminders goplaces	提醒事项管理 地点搜索与导航
AI/媒体	openai-image-gen canvas	OpenAI 图像生成 可视化渲染
系统	healthcheck model-usage session-logs	系统健康检查 模型使用统计 会话日志查看

类别	技能	功能
安全	1password	1Password 密码管理

23.2.3 Notion 技能深度分析

```
// Notion API v2025-09-03
// 术语："data sources"（原 databases）

// 支持的操作：
// - 页面 CRUD
// - 数据库查询（使用 data_source_id）
// - Block 内容管理
// - 属性类型：title, select, multi_select,
//   date, relation, rollup, formula, url, email...

// 限制：
// - 速率限制 ~3 req/sec
// - 需要处理 Retry-After 头
// - 数据库有双 ID:
//   database_id（创建时）vs data_source_id（查询时）
```

Listing 86: Notion 技能 API 特性

23.3 ClawHub 技能市场

技能通过 ClawHub (`clawhub.ai`) 平台分发：

- 社区贡献的技能可以发布到 ClawHub
- 用户通过 `openclaw plugins install <skill>` 安装
- 支持版本管理和自动更新
- 技能依赖声明在 `dependencies` 中（非 workspace 引用）

24 测试、构建与 CI/CD

24.1 测试策略

OpenClaw 采用多层次测试策略，覆盖率目标为 **70%**（行/分支/函数/语句）。

24.2 Vitest 配置矩阵

Table 39: 8 种 Vitest 测试配置

配置文件	范围	说明
`vitest.config.ts`	主配置	全局默认设置
`vitest.unit.config.ts`	单元测试	快速、隔离、纯函数
`vitest.gateway.config.ts`	网关测试	WebSocket、RPC 方法
`vitest.e2e.config.ts`	端到端	完整工作流
`vitest.live.config.ts`	实时 API	需要真实 API 密钥
`vitest.channels.config.ts`	渠道测试	渠道集成验证
`vitest.extensions.config.ts`	扩展测试	插件扩展验证
`ui/vitest.config.ts`	UI 测试	Web Components 测试

24.3 测试命名约定

Table 40: 测试文件命名规则

模式	类型	要求
`*.test.ts`	单元测试	与源码同目录，快速执行
`*.e2e.test.ts`	端到端测试	完整流程，可能较慢
`*.live.test.ts`	实时测试	需要 `OPENCLAW_LIVE_TEST=1`

24.4 测试工具模块

Table 41: 测试基础设施

模块	功能
`src/test-utils/`	通用 Mock 工具、断言辅助函数
`src/test-helpers/`	特定场景的测试辅助
`src/gateway/test-helpers.server.ts`	网关 Mock 服务器、请求上下文构建器、客户端模拟
`extensions/test-utils/`	扩展专用测试工具
`test-fixtures/`	静态测试数据和 Mock 响应

24.5 构建流程

```
# 主构建命令
pnpm build

# 构建步骤分解:
# 步骤 1: TypeScript -> JavaScript
#   tsdown 编译所有 .ts 文件到 dist/
#   生成 sourcemap

# 步骤 2: Plugin SDK DTS 生成
#   为每个 plugin-sdk 入口点生成类型声明文件
```

```
#   确保插件开发者获得完整的类型提示

# 步骤 3: 控制面板 UI 构建
#   Vite 打包 ui/ 目录
#   静态资源 (HTML/CSS/JS) 输出到 dist/ui/

# 步骤 4: Hook 元数据复制
#   复制 bundled Hook 的 HOOK.md 到 dist/

# 步骤 5: 构建信息生成
#   版本号、Git commit hash、构建时间

# 步骤 6: CLI 元数据生成
#   命令帮助文本、自动补全脚本
```

Listing 87: 完整构建流程

24.6 Docker 构建

24.6.1 多阶段构建

```
# 阶段 1: 提取扩展依赖
FROM node:22-bookworm AS ext-deps
# 提取所有 extensions/*/package.json

# 阶段 2: 构建
FROM node:22-bookworm AS build
# 安装 Bun (加速构建脚本)
# pnpm install (利用缓存)
# pnpm build
# 输出到 /dist

# 阶段 3: 运行时 (精简镜像)
FROM node:22-bookworm-slim AS runtime
# 不含构建工具和源码
# 以 node 用户运行 (非 root)
# 挂载卷: /home/node/.openclaw
# 健康检查: curl /healthz
EXPOSE 18789
```

Listing 88: Dockerfile 多阶段构建

24.6.2 Docker Compose

```
services:
  openclaw-gateway:
    build: .
    ports:
      - "18789:18789"     # 网关
      - "18790:18790"     # Bridge
    volumes:
      - config:/home/node/.openclaw
      - workspace:/workspace
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:18789/healthz"]
      interval: 30s
    restart: unless-stopped

  openclaw-cli:
    build: .
    depends_on: [openclaw-gateway]
    stdin_open: true
    tty: true
    environment:
      - OPENCLAW_GATEWAY_URL=ws://openclaw-gateway:18789
```

Listing 89: docker-compose.yml 服务

24.7 CI/CD 流水线

24.7.1 GitHub Actions 主工作流

Table 42: CI 任务矩阵

任务	说明
文档检测	检测是否仅文档变更（跳过重测试）
Node.js 测试	单元测试 + 集成测试
网关测试	WebSocket 协议测试
渠道测试	渠道集成测试
扩展测试	插件扩展测试
macOS 构建	Swift 应用编译和测试
Android 构建	Kotlin/Gradle 构建
Python 测试	技能脚本测试
Windows 测试	WSL2 兼容性验证
CodeQL 扫描	代码安全扫描
Docker 构建	容器镜像构建和发布

24.8 版本发布

Table 43: 版本发布策略

类型	格式	说明
稳定版	`vYYYY.M.D`	日期语义化版本
测试版	`vYYYY.M.D-beta.N`	Beta 预览版

发布渠道：

- **npm**：`latest`、`beta`、`dev` 标签
- **Docker Hub**：版本标签和 `latest`
- **macOS**：Sparkle appcast 自动更新
- **App Store**：iOS/Android 应用商店

24.9 守护进程管理

Table 44: 各平台守护进程

平台	机制	特性
macOS	launchd user agent	开机自启、崩溃自动重启
Linux	systemd user service	开机自启、日志集成
Windows	Scheduled Task (WSL2)	WSL2 内运行

25 设计模式总结与架构评价

25.1 核心设计模式

Table 45: 代码库中使用的 11 种核心设计模式

模式	应用位置	实现细节
适配器模式	渠道系统	每个渠道功能拆分为 10+ 独立适配器（Config、Security、Outbound、Messaging 等），通过组合实现完整渠道功能
工厂模式	依赖注入	`createDefaultDeps()` 和 `createPluginRuntime()` 创建带依赖的对象集，替代传统 DI 容器
注册表模式	插件系统	`PluginRegistry` 管理 8 种类型的注册项，支持动态注册和注销，按类型分组索引
观察者模式	Hook 系统	17+ 种生命周期事件，支持 Void（并行）和 Modifying（串行 + 累积）两种执行模式
策略模式	Memory 系统	嵌入提供商可插拔替换（OpenAI、Gemini、Voyage、Ollama 等），运行时选择搜索策略
模板方法	Memory 继承	`SyncOps → EmbeddingOps → IndexManager` 三层抽象，基类定义算法骨架，子类实现细节
命令模式	CLI 系统	Commander.js 命令树，每个命令封装为独立的处理器对象
中介者模式	网关	WebSocket 网关作为所有通信的中介，渠道和客户端不直接通信
队列模式	消息投递	文件系统持久化队列 + 指数退避重试 + 原子操作 + 崩溃恢复
组合模式	配置系统	多层配置层叠覆盖：.env → config.json → CLI → 渠道 Token
代理模式	Plugin SDK	SDK 封装底层实现细节，提供稳定的公共 API 表面

25.2 架构亮点

1. 可组合的插件系统

   8 种注册类型（工具、CLI、HTTP、渠道、提供商、Hook、服务、命令）覆盖了所有扩展场景。新功能可以完全通过插件实现，无需修改核心代码。Plugin SDK 的 40+ 导出入口点和完整的 TypeScript 类型支持，大幅降低了插件开发门槛。

2. 适配器关注点分离

   渠道功能拆分为 10+ 个独立适配器，每个专注于单一职责。新渠道接入只需实现必要的适配器，大幅减少了样板代码。适配器之间通过定义良好的接口通信，互不依赖。

3. 继承与组合并用

   Memory 系统通过三层继承实现代码复用（SyncOps → EmbeddingOps → IndexManager），同时提供接口（MemorySearchManager）支持替代实现。渠道系统则完全使用组合模式。两种模式各取所长。

4. 事件驱动 **Hook** 系统

   17+ 种生命周期事件覆盖从网关启动到 LLM 调用的全流程。Void Hook 并行执行提高吞吐量，Modifying Hook 串行执行保证一致性，优先级机制确保执行顺序可控。

5. 会话 **DAG** 模型

   消息通过 parentId 链形成有向无环图，而非简单的线性列表。支持对话分支、回溯、压缩和线程化展示，为复杂对话场景提供了灵活的数据基础。

6. 模块化网关方法

   RPC 方法从 30+ 领域处理器组聚合，每个处理器专注于特定领域。插件可以注册额外方法或覆盖现有方法，实现无侵入式扩展。

7. 多提供商容错

   Auth Profile 支持冷却、自动过期和轮换，配合故障追踪和预算控制，实现了高可用的 LLM 提供商管理。单一提供商故障不会影响整体服务可用性。

8. 文件系统持久化队列

   消息投递的三阶段原子操作（temp → rename → unlink）确保崩溃安全。指数退避重试策略在可靠性和资源消耗之间取得了良好平衡。

25.3 代码质量评价

Table 46: 代码质量维度评价

维度	评级	说明
类型安全	优秀	严格 TypeScript + Zod Schema 验证
测试覆盖	良好	70% 目标，8 种测试配置
模块化	优秀	77 个子模块，关注点分离清晰
可扩展性	优秀	8 种插件注册类型，40+ SDK 导出
文档	良好	Mintlify 文档站 + 内联注释
安全性	优秀	五层安全体系，审计日志
性能	良好	异步队列、缓存、防抖
可维护性	良好	一致的代码风格和命名约定

25.4 可维护性分析

优势：

- Monorepo 结构使跨包修改和重构更加容易
- 严格的 TypeScript 类型检查在编译时捕获大量错误
- Oxlint + Oxfmt 确保一致的代码风格
- 死代码检测（knip）防止代码膨胀
- Pre-commit Hook 在提交前自动检查

挑战：

- 部分核心文件过大（server.impl.ts ~38K 行，session-utils.ts ~28K 行）
- 77 个子模块的依赖关系需要仔细管理
- 多平台支持（TypeScript + Swift + Kotlin）增加了维护成本

25.5 扩展性评价

OpenClaw 的扩展性设计是其最大亮点之一：

- 纵向扩展：通过继承层次（Memory 系统）实现功能增强
- 横向扩展：通过插件注册（渠道、工具、Hook）实现功能增加
- 运行时扩展：动态加载/卸载插件，无需重启
- 协议扩展：网关 RPC 方法支持插件追加

25.6 与业界最佳实践对比

Table 47: 架构实践对比

实践	评价	说明
SOLID 原则	良好遵循	单一职责（适配器）、开闭原则（插件）、依赖倒置（接口）
12 Factor App	部分遵循	配置与代码分离、日志输出到 stdout、无状态进程
Clean Architecture	接近	分层清晰，但部分核心文件过大
Microservices 思想	适度采用	插件隔离，但非完全微服务
Event Sourcing	部分采用	JSONL 会话存储类似事件溯源

25.7 总结

OpenClaw 是一个架构精良、模块化程度极高的开源 AI 助手平台。其核心设计特征可以概括为：

1. 插件驱动：核心精简，功能通过 8 种注册类型的插件系统扩展
2. 网关中枢：WebSocket RPC 网关统一管理所有通信，配合 RBAC 和速率限制
3. 适配器组合：渠道功能拆分为 10+ 独立适配器，实现极高的关注点分离
4. 可靠投递：文件系统持久化队列 + 原子操作 + 指数退避，确保消息不丢失
5. 多层安全：从传输加密到路径边界、从凭据管理到审计日志，形成完整安全体系
6. 全面测试：8 种测试配置覆盖单元、集成、端到端和实时场景
7. 全平台覆盖：从 CLI 到 Web UI，从 macOS 到 iOS/Android
8. 开发者友好：完善的 Plugin SDK、丰富的内置技能、活跃的社区

该项目代表了一个生产级的 TypeScript Monorepo 系统，在可扩展性、安全性和可维护性之间取得了优秀的平衡。其插件优先的设计理念和模块化架构，为未来的功能扩展和社区贡献提供了坚实的基础。

联系作者

彦纾

wys17@tsinghua.org.cn

睿鸿

zhihong.lr@alibaba-inc.com

欢迎随意传播，我们支持技术的自由化